



Degree Project in Computer Science

Second cycle, 30 credits

Caching in Consensus Protocols for Low Bandwidth Replication

JOHAN EDELAND

Caching in Consensus Protocols for Low Bandwidth Replication

JOHAN EDELAND

Master's Programme, Software Engineering of Distributed Systems, 120 credits
Date: April 1, 2026

Supervisor: Harald Ng

Examiner: Paris Carbone

School of Electrical Engineering and Computer Science

Swedish title: Cachning i konsensusprotokoll för replikering med låg bandbredd

Abstract

Leader-based consensus protocols such as Raft are widely used in replicated databases and key-value stores. In Raft, a single leader is responsible for replicating every log entry to all followers. In geo-distributed deployments, this requires the leader to transmit all data over high-latency, bandwidth-limited links, making it a throughput bottleneck under higher load. Prior work has explored deterministic caching to reduce the replication traffic by encoding recurring fields. However, this approach is limited to leader-to-follower communication and requires an extra coordination phase to resynchronize caches after leader changes.

In this thesis, we introduce a caching mechanism for etcd Raft that requires no additional coordination between replicas. Caches are derived exclusively from committed log entries, ensuring that all replicas evolve identical state even through leader changes. To overcome the limitation of having the leader as the sole encoder, we add support for encoding by all nodes in the system. A key challenge is ensuring that data encoded by slow followers can be correctly decoded. To this end, a retention buffer is introduced. It holds recently evicted entries such that the leader can safely decode and recover the full data if it was encoded by a lagging follower.

The evaluation shows that the encoding scheme can be highly effective in multi-region settings. At high cache hit ratios, throughput improves significantly compared to unmodified Raft, with gains scaling with payload size. Compared to the design where only the leader encodes, the system improves throughput by up to 20%. These results show that deterministic caching can be integrated into Raft without requiring cache synchronization or restricting encoding to the leader, yielding significant throughput improvements for wide-area replication while preserving Raft's safety and semantics.

Keywords

Consensus, Raft, Caching

Sammanfattning

Ledarbaserade konsensusprotokoll som Raft används i stor utsträckning i replikerade databaser och nyckel-värdebutiker. I Raft ansvarar en enda ledare för att replikera varje loggpost till alla följare. I geodistribuerade miljöer innebär detta att ledaren måste överföra all data över länkar med hög latens och begränsad bandbredd. Detta gör ledaren till en flaskhals för genomströmning vid högre belastning. Tidigare arbete har utforskat deterministisk cachning för att minska volymen replikerad data genom att komprimera återkommande fält. Denna lösning begränsar dock komprimeringen till ledaren och kräver en extra koordineringsfas för att återsynkronisera cachar efter ledarbyten.

Detta arbete introducerar en cachningsmekanism för etcd Raft som inte kräver någon ytterligare koordination mellan replikor. Cachar härleds uteslutande från committade loggposter, vilket säkerställer att alla replikor utvecklar identiskt tillstånd även efter ledarbyten. För att ta bort ledaren som enda komprimerare kan vilken nod som helst som betjänar en klientförfrågan opportunistiskt komprimera förfrågningar. Ledaren gör en dekomprimeringskontroll innan replikering. Förslag återställs till fullständiga vid behov med hjälp av en retentionsbuffert som tillfälligt behåller nyligen utdrivna poster för att tolerera långsamma följare.

En multiregional utvärdering med skrivtunga arbetslaster varierar datastorlekar och träffandelar. Vid höga träffandelar ökar genomströmningen avsevärt jämfört med omodifierad Raft, med vinster som beror på datastorlek. Jämfört med en design där endast ledaren komprimerar förbättrar den utvärderade lösningen genomströmningen med upp till 20%. Fördelarna beror på datastorlek och träffandel, samtidigt som Rafts säkerhet och semantik förblir oförändrade.

Dessa resultat visar att deterministisk cachning kan integreras i Raft utan att kräva cachesynkronisering eller begränsa komprimering till ledaren, vilket ger betydligt förbättrad genomströmning för replikering över stora geografiska avstånd samtidigt som Rafts säkerhet och semantik bevaras.

Nyckelord

Konsensus, Raft, Caching

Acknowledgments

I would like to thank my supervisor Harald Ng for his guidance and continuous support during the project. His passion for distributed systems, shared by my examiner Paris Carbone, sparked my curiosity to explore the field further. I am also grateful to the friends I have made throughout this master's program, for the shared lunches, study sessions, and great moments together.

Stockholm, April 2026

Johan Edeland

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Scope	2
1.3	Methodology	2
1.4	Outline	3
2	Theoretical Background	5
2.1	Distributed Systems	5
2.2	Consensus	6
2.3	Replicated State Machines	8
2.4	Raft	9
2.4.1	System Model	9
2.4.2	Leader Election	9
2.4.3	Log Replication	9
2.4.4	Safety Properties	10
2.5	Omni-Paxos	10
2.5.1	Leader Election	11
2.5.2	Log Replication and Synchronization	11
2.6	Related Work	11
2.6.1	Geo-distributed Optimizations in MyRaft	12
2.6.2	Nilext-Aware Replication in Skyros	12
3	UniCache	13
3.1	Cache Construction and Mechanics	13
3.2	Eviction Policy and Correctness	14
3.3	Improving UniCache	15
3.4	Protocol Selection	15
3.4.1	Etd Raft	16
3.4.2	Etd Raft vs Omni-Paxos	16

4	Design	18
4.1	Design Challenges	18
4.1.1	Cache Consistency	19
4.1.2	Safe Encoding at Followers	20
4.1.3	Protobuf-Friendly Encoding	21
4.2	Overview Caching Module	21
4.2.1	Model	21
4.2.2	Cache Construction	22
4.2.3	Encoding strategy	22
4.2.4	Eviction and Retention	23
4.2.5	Proposal Safety	23
4.3	Cache integration Raft	23
4.4	Trade-offs	26
5	Implementation	27
5.1	Overview	27
5.2	Protobuf Encoding	28
5.3	Configuration	29
5.4	Propose Path	29
5.5	Leader Append Path	30
5.6	Follower Receive Path	32
5.7	Commit Path	33
5.8	Eviction and Retention	35
6	Experiments	37
6.1	Setup	37
6.2	Experimental Setup	38
6.2.1	Distributed Experiments	38
6.2.2	Workload	38
6.2.3	Compression Rate	38
6.3	Results	39
6.3.1	Impact of Forwarding Encoded Entries	39
6.3.2	Leader in majority region	41
6.3.3	Real World Workload	43
6.3.4	Compression	44
6.4	Leader Transitions	47
6.5	Microbenchmark Analysis	48
6.6	Profiling	52
7	Discussion	55

8	Conclusions and Future Work	59
8.1	Conclusion	59
8.2	Future Work	59
	References	61
A	Tables	67

Chapter 1

Introduction

Distributed systems have become an essential component of today's digital infrastructure, powering critical applications such as distributed databases, cloud services, and fault-tolerant storage platforms. These systems appear as a single system to its users, but in reality, they consist of multiple machines, connected through asynchronous communication links. This interconnection enables the building of highly available large-scale systems that can be accessed anytime, from anywhere. However, distributed systems face challenges not present in single-machine systems, most notably partial failures. Messages may be delayed or lost, nodes may crash, and clocks may drift. These issues complicate system design and introduce the risk of breaking the abstraction of a single coherent system.

To address these challenges, many distributed systems rely on consensus protocols. Consensus is a fundamental problem in distributed systems, concerned with getting multiple machines to agree on the same single value, despite failures and unreliable communication. Consensus protocols form the foundation for building replicated state machines, where a sequence of agreed upon values replicated across multiple machines constitutes a distributed log. This enables services that behave in a predictable way and to remain reliable under failures. Replicated state machines play a key role in the design of distributed databases [1, 2, 3], coordination services [4, 5], and large-scale file systems [6, 7].

Consensus protocols widely used in practice such as Multi-Paxos [8, 9, 10], Raft [11], and VR [12], rely on a single elected leader to coordinate data replication and ensure a consistent ordering of concurrent client requests across the system. By using a leader that orders and replicates all client requests onto the other nodes, the system is prevented from becoming

inconsistent due to different nodes handling requests in different order. However, this added responsibility introduces a substantial network bandwidth burden on the leader, since it must continuously send data to all other nodes. This can cause the leader to become a network bottleneck in the system. This is especially tangible in WAN environments, where nodes are distributed across regions and bandwidth is both limited and costly.

To address the leader bottleneck problem, recent work introduced UniCache [13], a deterministic caching mechanism integrated into the Omni-Paxos [14] consensus protocol. UniCache significantly reduces the leader's network load by caching frequently replicated entries and transmitting them in compressed format. While UniCache reduces the load on the leader, it has notable limitations. Followers that handle client requests must still forward the full data to the leader for replication. The forwarding of full data from followers to the leader introduces unnecessary network overhead that could have been avoided. Additionally, during leader transitions UniCache synchronizes caches in the prepare phase, which could potentially delay the accept phase due to more data having to be sent between nodes.

1.1 Problem Statement

This work will focus on the following two research questions:

How can UniCache be extended to support follower encoding?

How can caches be synchronized between nodes without sending the cache?

1.2 Scope

This thesis limits its scope to integrating UniCache into leader-based consensus protocols. The focus is to optimizing the network bandwidth usage. Other aspects such as storage or CPU optimization are outside the scope of this thesis.

1.3 Methodology

This thesis employs the quantitative research method, involving the following key steps:

1. Literature Review and Conceptual Adaptation: Studying foundational literature including Raft [11], OmniPaxos [14], and UniCache [13]. From this material, create an integration plan that preserves Raft’s safety and liveness guarantees.
2. Benchmark design: Design the benchmarking experiments for the different etcd/raft variants. Decide how the etcd/raft cluster will be deployed. Determine benchmark scenarios and which frameworks will be used to generate load.
3. Implementation: Extend the etcd/raft library [15] with caching, compression, and cache synchronization. Implement the experimental setup and benchmark scripts.
4. Data Analysis and Evaluation: Run the benchmarks, gather numerical data (throughput, network egress, hit-ratio), and compare both UniCache modes against unmodified Raft to identify gains and trade-offs.

1.4 Outline

The remainder of this thesis is organized as follows. Chapter 2 introduces the theoretical background required to follow the work, including Raft, Omni-Paxos, and related optimization techniques for leader-based replication. Chapter 3 presents UniCache, the deterministic caching mechanism that this work extends. Chapter 4 explains the design of RepliCache, covering cache consistency, follower-side encoding, and eviction safety. Chapter 5 describes the implementation, how the caching module was integrated into the *etcd/raft* library, and the automated benchmark and deployment framework. Chapter 6 presents and analyses the experimental results, comparing baseline Raft with both UniCache modes. Chapter 7 discusses the findings, their implications, and limitations. Finally, Chapter 8 summarizes the contributions and recommends directions for future work.

Chapter 2

Theoretical Background

In this chapter, the theoretical background for this work is presented. Section 2.1 defines distributed systems and the core abstractions and assumptions that govern their design. In 2.2 we formalize the consensus problem and briefly review Paxos. Section 2.3 introduces replicated state machines and the role of consensus in log-based replication. We then turn to Omni-Paxos and the Raft algorithm in section 2.5 and 2.4. Finally, Section 2.6 covers related work.

2.1 Distributed Systems

A distributed system is a collection of independent components or nodes that work together to achieve a common goal, appearing to users as a single, unified system [16, 6]. Each node of the system is a separate machine or server that operates on its own but communicates with the other nodes of the system via communication links. Messages sent through these links may be delayed, dropped, reordered, or duplicated.

A node of a distributed system can fail in the following ways:

1. **Crash-stop:** The node fails by crashing, stopping all its processing and communication, never recovering.
2. **Omissions:** The node omits sending or receiving messages.
3. **Crash-recovery:** The node fails by crashing, stopping all its processing and communication. The node may later recover its state, meaning it needs to have access to some storage device to which it can write/read its state.

4. **Byzantine:** The node can behave in any arbitrary way, not just crashing. This can include sending incorrect data, updating its state incorrectly, or behaving maliciously, attacking the system or deceiving other nodes of the system.

In this work, we assume the crash-recovery model.

The timing assumptions of distributed systems can be synchronous, meaning there is an upper time-bound on communication, it can also be asynchronous (no bounds), and lastly partially synchronous [17] which is a combination of the two. The partially synchronous model exhibits no time-bound initially but will eventually go through periods of synchrony where nodes can synchronize and make progress. In this work, we assume the partially synchronous model, which is a common assumption in distributed systems.

Two key properties of distributed systems are Safety and liveness. Safety guarantees that nothing bad ever happens, while liveness guarantees that something good eventually will happen. In practice, many protocol specifications adopt the asynchronous, crash-recovery model for safety proofs [8]. However, in a purely asynchronous system deterministic consensus cannot guarantee termination [18], so liveness is usually argued under partial synchrony.

In summary, distributed systems must cope with unreliable networks and partial failures while coordinating independent nodes. These challenges make them more complex than single node programs, but they also enable scalability and fault tolerance: the system can continue operating as long as enough nodes and links remain.

2.2 Consensus

Consensus is the problem of getting a group of independent, possibly unreliable processes to agree on a single value despite crashes, network delays, or message loss. Consensus protocols are often specified by four formal guarantees [9, 8]:

- **Validity:** Any value that is decided must have been proposed by some node.
- **Agreement:** All correct nodes must decide on the same value
- **Integrity:** A node never changes its decision. It can only decide once.

- **Termination:** Every correct node eventually decides some value

Paxos was the first protocol to satisfy these properties. It was introduced by Leslie Lamport in The Part-Time Parliament [9, 8], and later clarified in Paxos Made Simple to clarify parts of the protocol. In Paxos, proposers propose values, acceptors vote on them, and learners learn the final agreed value. Consensus deployments consist of a collection of N nodes communicating over an asynchronous network. Protocols in this family (e.g. Raft [11]) can tolerate at most $f = \lfloor (N - 1)/2 \rfloor$ failures, so a cluster of five nodes will continue operating with up to 2 failed nodes. This works by requiring a quorum, any majority subset of nodes, meaning at least $\lfloor N/2 \rfloor + 1$ to agree on a value. Since any two majorities intersect, safety is preserved and the system can make progress whenever a quorum is available.

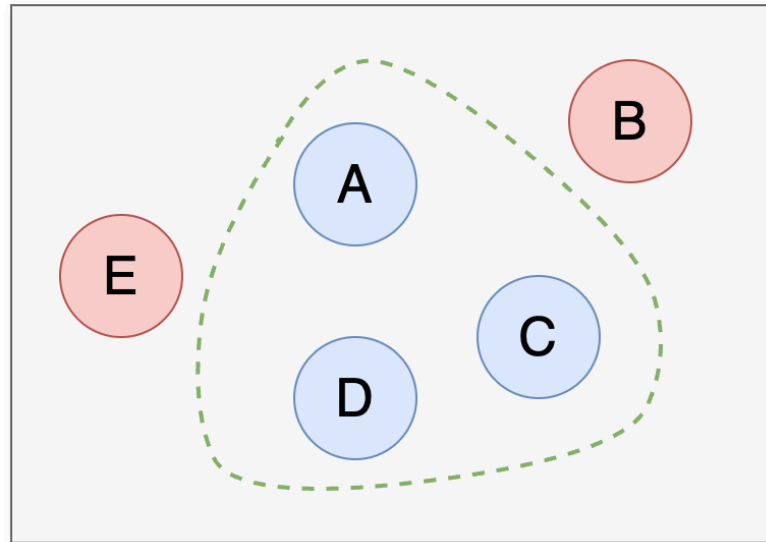


Figure 2.1: A cluster of 5 nodes can make progress as long as a majority of nodes are active and communicating. In this example A, C, and D form a quorum, while nodes B and E are unavailable.

Consensus algorithms ensure that a distributed system remains both fault tolerant and consistent by coordinating agreement on shared state despite node crashes, dropped messages, or network partitions. They keep all copies of data in sync in distributed databases like CockroachDB [1], maintain the integrity and ordering of blocks in blockchain systems [19], and guarantee metadata consistency in large-scale file systems like Google File System [6].

2.3 Replicated State Machines

A replicated state machine is a design pattern for achieving strong consistency and high availability in distributed systems. Each replica maintains the same deterministic state that evolves identically, given the same sequence of commands. The sequence of commands is created by clients submitting operations that are first agreed upon via a consensus protocol, before being applied to the state machine. Because every command is replicated before execution, replicas stay in sync and the system continues operating even if individual nodes fail [20, 16].

Paxos-based RSMs extend the basic Paxos protocol by using a replicated log to ensure all replicas apply the same commands in the same order. Most approaches use an elected leader to drive the replication while some adopt a leaderless model.

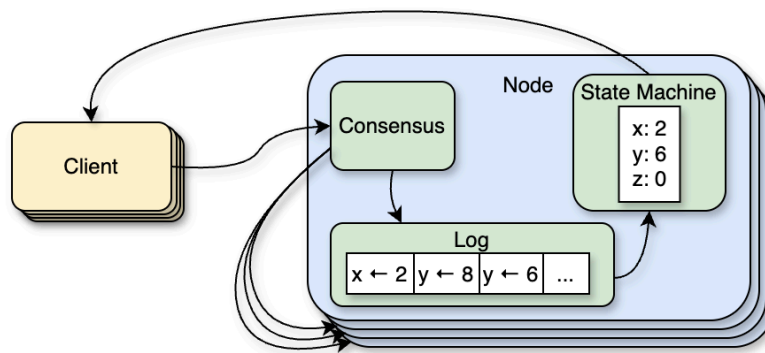


Figure 2.2: Replicated State Machine. Clients send commands to nodes, which run a consensus protocol (e.g. Paxos or Raft) to manage a shared log of operations. Once an operation is committed in the log, each node applies it to its local state machine in the same order, ensuring all replicas to produce identical states and respond consistently to clients.

Among leader-based replicated state machines, one of the most widely adopted algorithms is Raft [11]. Raft was motivated by what the authors saw as two shortcomings of Paxos: the complexity of the protocol, and the lack of a widely agreed-upon multi-decree replication algorithm. Raft addresses these issues by providing a clear, end-to-end protocol for state machine replication. It covers leader election, log replication with terms and indexes, and safe membership changes while preserving the same safety and liveness guarantees as Paxos-based RSMs [11].

2.4 Raft

Raft is one of the most widely adopted consensus protocols in practice, powering systems such as etcd [21], Dgraph (distributed graph database) [22], CockroachDB (distributed SQL) [1], TiKV [2], and RethinkDB [23]. Its clarity and robustness have made it a popular choice for metadata management, leader election, and replicated state machines in production environments. Raft decomposes the consensus problem into three distinct concerns: Leader election, log replication, and safety.

This section reviews Raft's system model, roles of nodes, election, log replication and safety guarantees.

2.4.1 System Model

Each Raft node maintains a set of persistent variables keeping track of the current term, which node the node last voted for and most importantly the log, storing the replicated entries. The nodes also store index of the highest committed entry and the last entry applied to the local state machine in volatile memory. In addition to this, the leader keeps track of the highest known index to be replicated as well as the next index to send to each follower.

The nodes in a Cluster can take on the roles of *leader*, *follower*, or *candidate*. Under normal operation of the Raft cluster there is exactly one leader, which is responsible for driving the replication. Followers passively await and respond to messages from leaders or candidates. The candidate state is used when followers notice a passive leader and initiates a new election.

2.4.2 Leader Election

Leader election in Raft is based on randomized timeouts. If a follower times out without hearing from a leader, it becomes a candidate, increments its term, and requests votes from other nodes. Each node grants at most one vote per term, and only to a candidate whose log is at least as up to date as its own. This ensures that a newly elected leader always has the most complete log. If elections split the vote, timeouts trigger new rounds until a leader emerges [11].

2.4.3 Log Replication

Once elected, the leader handles requests by appending commands to its log and replicating them to followers. Each replication includes the leader's term,

the index and term of the log entry preceding the new ones, the new entries to be replicated, and the leader's commit index. Followers accept the entries only if the preceding log entry matches, otherwise they reject and the leader retries with an earlier index. This mechanism guarantees log consistency: followers either align with the leader or roll back conflicting entries. An entry becomes committed once it is stored on a majority of nodes and belongs in the leader's current term. It can then be applied to each node's local state machine. learns that an entry is committed, it will apply that entry to its state machine. [11]

2.4.4 Safety Properties

Raft guarantees that no two nodes apply different commands at the same log index. Key invariants include:

- **Election Safety:** there can only be at most one elected leader in a given term.
- **Leader Append-Only:** a leader only appends new entries to the log, it never overwrites or deletes existing entries.
- **Log Matching:** if two logs contain an entry with the same term and index then all preceding entries are identical.
- **Leader Completeness:** a leader contains all committed entries in its log at the time it becomes leader.
- **State Machine Safety:** a server applies entries in log order and never applies different entries at the same index.

Together, these rules ensure that all committed entries survive leader changes and that the replicated state machines remain consistent [11].

2.5 Omni-Paxos

Omni-Paxos is a Replicated State Machine designed to tolerate partial network partitions, situations where a majority quorum remains reachable only through intermediate nodes. The protocol does this by splitting *leader election*, *log replication*, and *reconfiguration* into three different components, each optimized in isolation. This design yields both strong safety and liveness under partial connectivity, and also enables the protocol to do parallel reconfiguration. [14]

2.5.1 Leader Election

Omni-Paxos extends leader election to work under partial network connectivity by requiring only a quorum-connected server to make progress. Its Ballot Leader Election uses periodic heartbeat carrying nodes ballot and a quorum-connected flag in order for other nodes to detect liveness and QC status. If a follower suspects the leader is gone it increments its ballot and gathers promises, the candidate that obtains most promises from a quorum connected majority becomes the new leader [14].

2.5.2 Log Replication and Synchronization

The log replication protocol used in Omni-Paxos is Sequence Paxos, which satisfies the properties of *Sequence Consensus*: validity, uniform agreement, and integrity. Sequence Paxos divides log synchronization and log replication into two phases, *Prepare* and *Accept*.

The prepare phase is critical for maintaining log consistency when a new leader has been elected. The newly elected leader collects `Promise` messages from a majority of nodes. Each `Promise` contains the node's highest accepted ballot and the length of its current log. The leader uses this information to identify the most up-to-date log among its followers. It then sends `AcceptSync` messages to bring all followers up to date before it can begin replicating new log entries in the accept phase.

After the prepare phase, the accept phase begins. Client requests are issued via a `Propose` message. The leader appends each command to its log and replicates it to its promised followers by sending `Accept` messages. Followers append the entries to their log reply with `Accepted`. Once the leader receives a majority of `Accepted` for a given index, it sends `Decide` to its promised followers to let them know the decided (committed in Raft) index has been updated.

This separation of log synchronization and log replication enables even nodes lagging behind to be elected as leader, as long as it is quorum connected. [14]

2.6 Related Work

A variety of optimizations have been proposed to enhance the performance of consensus protocols, focusing primarily on reducing replication latency and

improving throughput under high-load or geo-distributed deployments. Two notable examples are MyRaft and Skyros.

2.6.1 Geo-distributed Optimizations in MyRaft

Developed by Meta, MyRaft integrates Raft into their existing MySQL server, replacing its built-in semi-synchronous replication with the fault-tolerant consensus-driven architecture of Raft [24]. To meet Meta's strict latency and availability targets across geo-distributed replicas, the team introduced two key Raft enhancements. The first, **FlexiRaft**, minimizes cross-region latency by allowing operators to configure region-local commit quorums at startup. The second, **Proxying**, reduces load on the leader by routing replication traffic bound for remote regions through a proxy: the leader sends each `AppendEntries` RPC only once to the proxy, which then fans it out to the replicas in that region, while still proxying acknowledgements back to the leader [24]. In production, MyRaft reduced the window of unavailability after a primary failure from about one minute down to just a few seconds, while preserving commit latencies on par with native MySQL replication [24]. These optimizations demonstrate how carefully tailored quorum selection and network-aware routing can adapt a general-purpose consensus protocol to the demands of large-scale, low-latency OLTP (Online Transaction Processing) workloads.

2.6.2 Nilext-Aware Replication in Skyros

Another line of optimization for log replication is SKYROS, a “nilext-aware” RSM developed by VMware Research. Much like UniCache exploits recurring data fragments, SKYROS leverages a different semantic insight: many storage-layer operations never externalize their effects (nil-externality). By immediately making such “nilext” updates durable in a lightweight per-replica log and deferring their formal ordering and execution until a later read or non-nilext write would expose them, SKYROS completes these operations in one round-trip rather than two [25]. In YCSB microbenchmarks, it achieved up to 3x lower latency compared to a batched Paxos baseline, and when wrapped around RocksDB [26] it yielded similar geo-distributed replication speedups without sacrificing linearizability [25]. This demonstrated that, by exposing interface-level semantics rather than treating commands as opaque blobs, one can harvest significant “free” performance gains in replicated state machines, just as UniCache [13] does via caching.

Chapter 3

UniCache

UniCache extends traditional leader-based log replication protocols by exploiting recurring patterns in application workloads to reduce the amount of data transmitted over the network. The central idea is that many commands replicated across state machines share common structure or values, such as SQL templates or frequently recurring attribute values. Rather than repeatedly transmitting these values in full, UniCache maintains a cache at each replica, mapping common fragments to compact integer identifiers. When the leader prepares a command for replication it replaces cacheable fragments with their identifiers, yielding a compressed representation. Followers receiving such messages decode the identifiers using their local caches, reconstruct the original command, and append it to their logs. Since caches evolve identically across replicas, this compression is transparent: the replicated log remains unchanged, and the safety and liveness guarantees of the protocol are preserved [13].

3.1 Cache Construction and Mechanics

The mechanics of UniCache are built around a local cache at every replica, implemented as a bidirectional map between encodable data fragments and integer indices. UniCache does not attempt to cache every field, as some fields such as timestamps, UUIDs or other unique identifiers rarely repeat and thus provide no benefit. Instead, the developer specifies which fields or structures within commands are eligible for caching. For example, the template of an SQL query or the constant values in a CSV column may be cached, while unique attributes always remain as is.

When a leader receives a new command from a client, it applies a

preprocessing step that splits the command into encodable and non-encodable parts. Each encodable fragment is then replaced with its cached identifier if it exists in the local map. The leader then issues an `AcceptEncoded` message to its followers containing the both encoded and unchanged data. Upon receiving this message, followers decode the identifiers using their local caches, reconstruct the full command and append it to their logs. This process ensures that UniCache alters only the representation of the data transmitted over the network without modifying the underlying consensus protocol.

3.2 Eviction Policy and Correctness

The central question of correctness is how UniCache guarantees that encoded values sent by the leader can always be decoded by the followers. The answer lies in the fact the caches evolve deterministically from the sequence of log entries replicated at each replica. If every replica applies the same deterministic update and eviction rules to an identical sequence of inputs, their caches remain consistent, just as their logs do. This makes the cache effectively another deterministic state machine layers on top of the replicated log [27].

However, caches in UniCache are updated optimistically. In the implementation, encodable items are inserted into the cache as soon as their entries are appended to the log in the accept phase, before they are actually committed. This design improves compression efficiency because it allows UniCache to exploit patterns immediately rather than waiting for commit, but it also introduces the risk of cache drift. If the leader fails after sending accept messages that later turn out not to be committed, some replicas may have updated their caches with items that other have not.

To resolve this, UniCache relies on Omni-Paxos log synchronization mechanisms. When a new leader is elected, Omni-Paxos runs the previously mentioned prepare phase. UniCache extends this mechanism by also transmitting the most up-to-date cache along with the missing log entries in the `AcceptSync` messages. In this way, any divergence caused by optimistically appending to caches is eliminated after each leader election without compromising the safety of the protocol.

UniCache adapts workload changes through using any deterministic cache eviction policy fitting of the application workload. Static policies such as Least Recently Used (LRU) or Least Frequently Used (LFU) can be employed to manage cache entries, but these may not perform well when workloads shift over time. To address this, UniCache also supports an adaptive reinforcement learning-based policy, LeCaR [28], which dynamically balances between

LRU and LFU. This allows UniCache to maintain high cache efficiency under workloads with varying temporal or frequency skews, ensuring that compression benefits are preserved even as the workload changes.

3.3 Improving UniCache

UniCache introduced caching to Omni-Paxos and demonstrated substantial throughput improvements, particularly in high-latency environments and with larger payloads. Its implementation synchronizes caches during the *Prepare* phase, ensuring replicas converge after leader changes. While effective within Omni-Paxos, this design relies on protocol-specific coordination and optimistic cache updates tied to the accept phase.

To explore a more general caching approach, independent of these mechanisms, two paths are possible: modify UniCache within Omni-Paxos to remove its reliance on the Prepare phase, or re-implement the mechanism in another leader-based protocol. The latter option also offers the opportunity to evaluate the technique in a widely adopted system.

3.4 Protocol Selection

Extending the existing solution in Omni-Paxos would be advantageous in the way that it would be a less complex task than integrating caching from scratch in another protocol. It would only require the solution to find a way to synchronize caches other than using the Prepare phase. To implement caching in another protocol would be more complex as a completely new solution would need to be designed.

However, there are advantages to implementing the solution in another protocol. It would not only enable for comparison with the existing solution, but also an opportunity to implement the solution in a protocol that is widely used by the industry. This would make the findings of this work more applicable to real-world systems, increasing its relevance and the potential impact of the research.

Raft is one of the most widely adopted consensus protocols in the industry, powering many distributed systems in use today (etcd, CockroachDB, Apache Kafka, blockchain systems). Therefore, it makes a strong candidate for the implementation of this work.

3.4.1 Etcd Raft

There are many open source raft protocols to choose from. Etcd raft is one of the most popular ones. Etcd is the maintainer of the library and uses it for its distributed key-value store, which in turn is used by Kubernetes to store critical information such as configuration data, state data, and metadata. CockroachDB is another system that builds on top of etcd raft. Another reason for choosing this library is that it's well documented and has an active community, which should reduce the risk of encountering any unexpected bugs.

3.4.2 Etcd Raft vs Omni-Paxos

Before designing the solution, it is important to understand the largest differences in terms of the architecture of etcd raft and Omni-Paxos and how these affect the implementation of caching in etcd raft.

The servers in Omni-Paxos build their caches optimistically from the accepted entries, which are entries not yet decided. This can be a problem if the leader tries to replicate an entry but before it's replicated on a majority of nodes, the leader fails and the entry is never decided. When this happens, some servers may have added the encoded entry to their cache, while others never received and therefore never adds it to their cache, meaning the caches become out-of-sync. The existing solution solves this by utilizing the prepare phase, which makes sure that all servers have the latest log, by also sending the cache corresponding to the latest log.

In the current OmniPaxos implementation, the leader node explicitly handles all client requests, appending each request to its log and replicating entries to followers. OmniPaxos does not forward client proposals from followers to the leader during the Accept phase. As a consequence of this, the existing UniCache implementation in Omni-Paxos performs all encoding exclusively at the leader.

Raft does not have this separation of leader election and log replication. The elected leader always has the most up to date log. This means as soon as a new leader is elected, no synchronization of the log is needed. Furthermore, the etcd raft library supports any node serving client requests. All nodes handle client requests in the same way regardless of the role of the node. The data from the request is forwarded to a proposal channel, from which the leader retrieves data to be replicated.

These differences have implications for the design of the cache in raft. First and foremost how do we synchronize the cache without the prepare phase?

Secondly, we can make use of the fact that any node can serve client requests in etcd raft. By enabling any node to propose encoded entries we offload the leader of the burden of encoding all entries.

Chapter 4

Design

4.1 Design Challenges

As mentioned in the previous chapter, adapting UniCache to Raft introduces several challenges arising from differences in how Raft manages log progression, client proposals, and leader transitions. In Omni-Paxos, UniCache relies on the *Prepare* phase to both reconcile optimistic cache updates and ensure that all replicas share identical cache state. Since Raft lacks a comparable phase, cache correctness, eviction behavior, and follower-side encoding requires new mechanisms.

The proposed solution, RepliCache, shifts the responsibility of cache synchronization from an explicit coordination phase to the inherent determinism of the Raft log. By deferring all cache updates until an entry is officially committed, we ensure that every replica derives its cache from the same immutable sequence of data.

Furthermore, to maximize the benefits of this model in high-load environments, we decentralize the encoding process. Unlike the original UniCache, which limits encoding to the leader, RepliCache allows any node to encode client proposals before forwarding them. The leader then acts as a final validator, retaining evicted entries for a safe interval of committed entries to ensure that all encoded proposed entries will be decodable by even the slowest followers by the time it is received. This integrated approach is detailed in the following sections.

4.1.1 Cache Consistency

Since raft doesn't have a Prepare phase, another way to synchronize the caches between nodes is needed. A straightforward way of doing that would be to build the cache only out of committed entries. In the UniCache paper, it is stated that caches are built from committed entries. With this approach, the cache needs to be checked twice; once when receiving an entry to decode, and a second time once it is committed to update the cache. In the implementation of UniCache, an optimization to this approach is employed, which is building the cache optimistically from unstable entries. This optimization means caches only need a single check per entry, when it is received, where both decoding the entry and updating the cache is taken care of. However, this can lead to caches becoming out of sync if entries are added to caches but never committed. This may happen for example on leader failures.

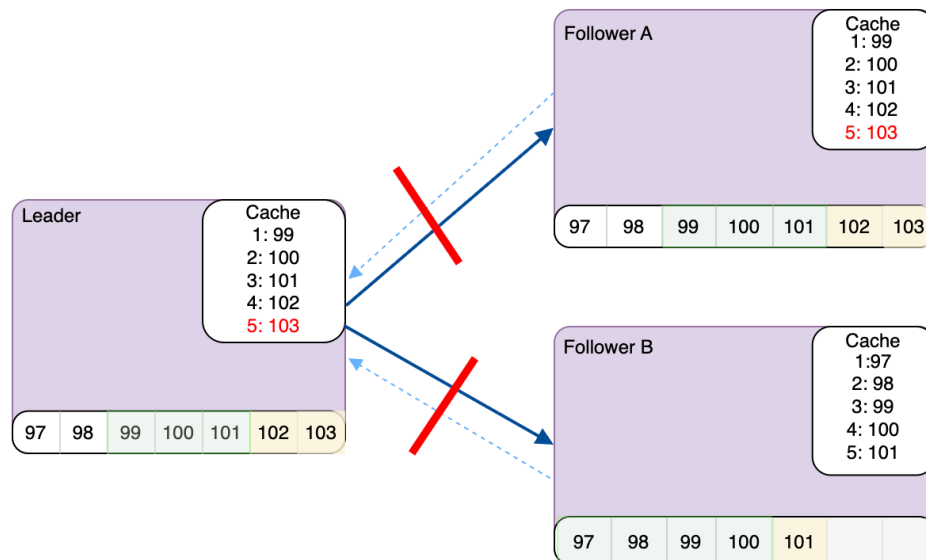


Figure 4.1: Problem with cache consistency when optimistically adding to caches.

Figure 4.1 shows the problem with optimistic caching (adding uncommitted data to the log). In this scenario, the leader replicates entry 103 and updates its local cache. Follower A receives and caches this entry, while Follower B remains at index 102. If the leader fails before entry 103 is committed, a new leader, starting from the last now known commit index of 101, will overwrite index 103 with a different command. Consequently, Follower A's cache now

contains an invalid cache entry that does not exist in the final replicated log, leading to divergence in cache states between replicas.

There are alternative ways of solving this problem. One option is to also add entries optimistically, but this solution would require us to come up with a rollback mechanism on all entries that were added but never committed. This would likely mean that we need to introduce additional coordination similar to the prepare phase, which is what we want to avoid.

This work will aim to solve this problem by building the cache from committed entries, utilizing the deterministic properties of the log. In raft, the leader election guarantees that the new leader has the most up to date log. The leader will therefore by definition also have the most up to date cache, eliminating the need for synchronizing caches.

4.1.2 Safe Encoding at Followers

Encoding entries at follower nodes introduces additional challenges in keeping the protocol consistent. If we derive the cache from the committed entries of the log, it will remain consistent across nodes. However, nodes may progress at different rates, meaning at any given time, a follower may not have advanced as far as the leader or other followers. Two issues arise from this:

1. **Eviction before replication:** A follower may handle a client request and encode an entry while it is still in its cache. By the time the leader retrieves the entry and tries to replicate it, system progress may have caused the entry to be evicted, which would result in the leader's inability to replicate the entry.
2. **Eviction during transmission:** The leader might still have the entry in its cache and send it in an `AppendEntries`, but a follower may evict the corresponding entry from its cache before receiving the encoded `AppendEntries`. This results in the follower not being able to decode the entry and replication fails.

Since only the leader node has the complete view of the state of all nodes in the system, it is the only node capable of accurately determining if replicating a particular encoded entry is safe. Therefore, the leader will keep evicted entries for a period of time before removing them completely. By keeping evicted entries, the leader will be able to restore proposed encoded entries that have either already been evicted, or risk being evicted before arriving at followers.

4.1.3 Protobuf-Friendly Encoding

The etcd Raft library uses gRPC for communication, which involves serializing data into protobuf format before transmission. In the UniCache design for Omni-Paxos, the encoding and decoding operations are straightforward: encoding replaces the value of an object field with a compressed representation, and decoding restores the original value. These operations assume direct access to deserialized objects.

However, when implementing UniCache in the etcd Raft library, we face a new challenge. Because data transmitted via gRPC is already serialized into protobuf byte arrays, directly modifying object fields is not straightforward. Normally, altering serialized data would require fully deserializing the protobuf messages, modifying fields, and then serializing again, incurring significant overhead and negating some performance benefits of UniCache.

To address this, specialized functions capable of directly manipulating serialized protobuf data will be implemented. The purpose of these functions would be to efficiently scan the byte arrays of serialized protobuf messages, allowing extraction and replacement of individual field values and wire types without full deserialization. Developing these functions involves carefully handling protobuf encoding rules, managing variable-length encoding, and ensuring the rules of protobuf encoding isn't broken.

This serialization-aware encoding mechanism introduces additional complexity into our UniCache implementation, but it is essential to achieve meaningful performance benefits without the overhead of repeated serialization and deserialization cycles.

4.2 Overview Caching Module

We extend the etcd Raft library with a caching module that lets any node propose compressed data while preserving Raft's safety and keeping the library API unchanged.

4.2.1 Model

The cache module should be instantiated with the parameters capacity, field number of the protobuf byte array to cache, and minCacheVersion(), a function callback that returns the minimum commit index any follower has incorporated into its cache.

Each key (encoded ID) in the cache is mapped to the full bytes of the cached field, the index for when the field was added to the cache and the index of its last use in the log. The callback function `minCacheVersion()` is used by the leader to keep track of the cluster progress.

4.2.2 Cache Construction

Caches are derived from committed entries only. When an entry is committed at index i , if it carries an encodable field that is previously unseen, the field is added to the cache and map it to a new ID. If the field is already present, the LRU and the keys last used index is updated.

To avoid redundant key extraction on the commit path, each entry carries an `EncodedID` field which contains the cache identifier assigned during `EncodeData`. When `EncodedID` $\neq 0$, the committing node can look up the cache entry directly by integer ID rather than re-extracting the protobuf field bytes.

4.2.3 Encoding strategy

Any node may encode data from client requests before proposing by substituting the encodable field of the protobuf array with the mapped ID. The safety of follower-side encoding rests on the determinism of cache construction. Because every replica builds the cache by processing committed log entries in the same order, applying the same LRU insertion rules, each replica assigns identical integer IDs to identical data fragments. A follower can only have ID i for key k in its cache if the log entry that introduced k has been replicated and committed by the leader. As a result, any ID a follower uses when encoding a proposal is an ID that the leader's cache also contains, with the same key-to-ID mapping.

Before sending `AppendEntries` to followers, the leader evaluates if the entry is safe to send in encoded form or if the entry needs to be restored. The condition for replicating an entry in encoded form is:

$$\begin{aligned} \text{appendIdx}(\text{key}) - \text{lastIdx}(\text{key}) &\leq \text{capacity}(\text{cache}) \\ \&\& \\ \text{minCacheVersion}(\text{followers}) &\geq \text{addedIdx}(\text{key}) \end{aligned}$$

The first condition ensures the entry won't get evicted before arriving at the followers for decoding. The second condition ensures that the cache of all followers already contains the entry. If the check fails, the leader falls back

to sending the full entry. The leader also retains a buffer of recently evicted entries to be able to restore entries whose IDs were just evicted.

4.2.4 Eviction and Retention

The module uses a Least Recently Used (LRU) eviction policy for the cache. When inserting a new key and the cache has reached capacity C , the least recently used key is evicted from the cache.

To account for followers lagging behind (i.e., with a lower cache version than the leader), the leader does not discard evicted entries immediately. Instead, those entries are kept in a bounded retention buffer which is purged once the buffer has grown to a pre-specified size.

4.2.5 Proposal Safety

When a follower encodes a proposal using a cached key, the key's mapping must still be resolvable by the leader at the time it appends the entry to its log. If the key has been evicted from the leader's active cache and also purged from the retention buffer, the original data cannot be recovered. Since replicating an unresolvable entry would cause followers to fail decoding, the leader must reject the proposal entirely.

When `SafeEncode` returns `nil` for an entry that carries a non-zero `EncodedID`, the leader drops the entire proposal batch rather than silently falling back to the full payload. This is a deliberate safety measure: by the time `SafeEncode` is called, the entry's `Data` field still contains the encoded form from the proposing node. Without the original key bytes available in either the active cache or the retention buffer, there is no way to restore the full entry. Dropping the proposal forces the client to retry, at which point the proposing node will either send the entry raw (if the key has also been evicted from its own cache).

4.3 Cache integration Raft

Clients can send requests to any node in the system, including followers. Upon receiving a client request, the node invokes the `EncodeData` function. This function checks if the request data already exists in the local cache. In the case of a Cache Hit, the data is replaced by an encoded (compressed) version to reduce bandwidth usage. A cache miss results in the data being proposed in its original form.

The node then proposes the resulting data (encoded or original) by placing it into the Raft library's internal propose channel. Along with the proposal, the node explicitly indicates which field of the serialized buffer has been encoded, to assist the leader in correctly parsing the proposal.

The leader retrieves proposed data from the propose channel and appends it to its local log as entries. To replicate these entries to followers, the leader generates a slice of the log for each follower, starting from the respective follower's minimum index (*minIndexFollower*) onwards.

Before replicating, the leader performs a safety check to ensure encoded entries are safe to send to followers. This check verifies that:

- The entry has already been learned (cached) by the follower.
- The encoded entry will not be evicted before arriving at the follower.

If the leader determines an entry isn't safe to send encoded, it restores the entry to its original form before replication.

Followers receiving entries to be replicated run the `DecodeEntry` function for each received entry. This ensures that all entries appended to a follower's local log are stored in their fully decoded (original) form, maintaining log consistency and avoiding partial or encoded entries in the persisted logs. After having appended received entries, followers respond to the leader with their current cache version. The leader updates the followers cache version and use it for calculating the "safe window" of encoded entries.

Once entries are committed (learned) across the cluster, all nodes including the leader loop through each newly committed entry, updating their local caches. Since the consensus protocol guarantees deterministically ordered log entries across all nodes, this ensures caches stay consistent between nodes. After processing the committed entries, nodes update their cache version.

Additionally, the leader periodically purges any temporarily stored evicted entries that are no longer needed, optimizing memory usage.

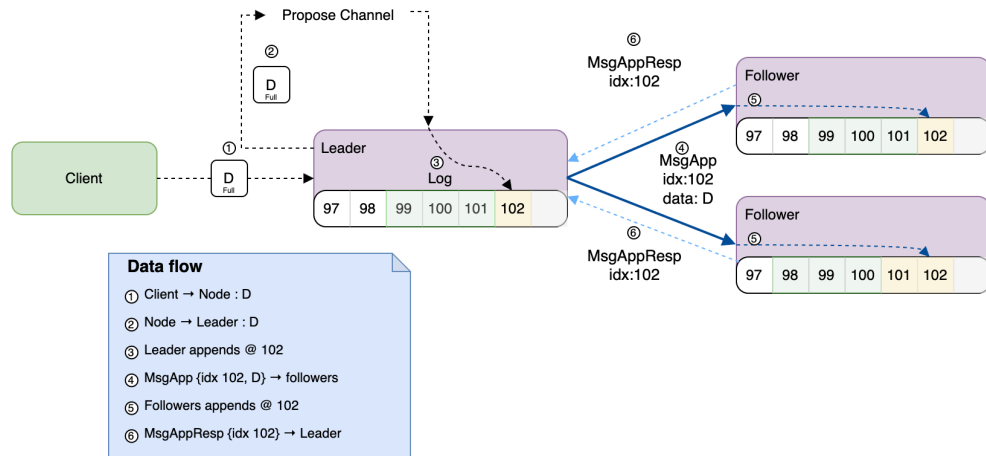


Figure 4.2: Data flow of etcd/raft without any caching mechanism implemented. (1) The client sends data D to a node. (2) The node forwards the data to the leader. (3) The leader appends the entry to its log at index 102. (4) The leader sends an AppendEntries message {idx 102, D} to all followers. (5) Each follower appends the entry at index 102. (6) The followers reply with a MsgAppResp {idx 102} acknowledgment back to the leader.

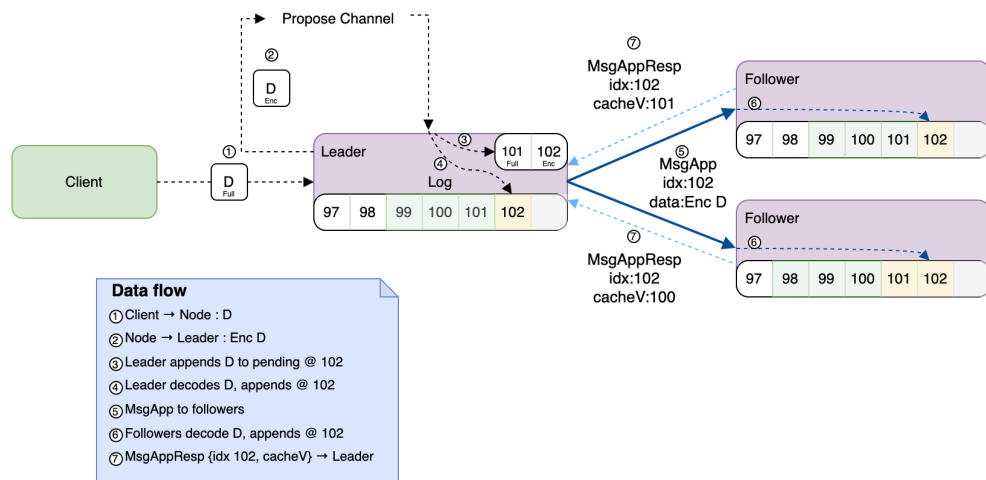


Figure 4.3: Data flow when caching is enabled. (1) The client sends data D to a node. (2) The node checks its local cache and encodes the data if a cached entry is found, then forwarding the entry to the leader. (3) The leader appends the encoded entry to its log. (4) The leader sends an AppendEntries message {idx 102, Enc(D)} to all followers. (5) The followers decode the entry using their cache and append the decoded data at index 102. (6) The followers reply with a MsgAppResp {idx 102, cacheV: 102} acknowledgment back to the leader.

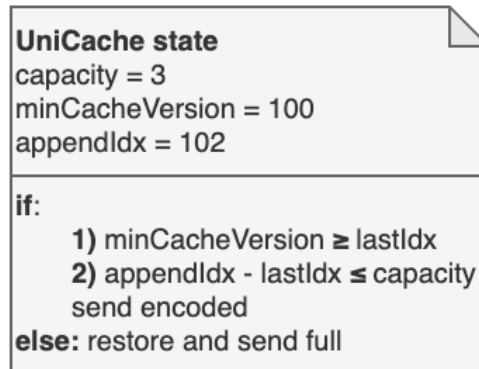


Figure 4.4: Leader state variables used to determine whether the entry proposed is safe to replicate as encoded or if fallback to full is needed. Used in step 4 before appending to pending buffer.

4.4 Trade-offs

This approach uses a window where it is considered safe to replicate encoded entries by introducing a retention buffer. This buffer is used to restore entries that have been evicted but may still be referenced by a follower encoding a proposal. To prevent unbounded memory growth, the retention buffer is capped at a configurable size specified by the user. When the buffer is full, the oldest evicted entries are purged, regardless of follower progress. If a follower proposes an encoded entry whose ID has been purged from both the active cache and the retention buffer, the leader cannot restore the original data. In this case, the proposal is dropped and the client must retry. This bounds the leader's memory at the cost of rejecting stale encodings from followers that have fallen too far behind.

Chapter 5

Implementation

5.1 Overview

This chapter describes the integration of UniCache into the etcd/raft library. The goal of the implementation is to introduce caching without affecting the Raft API or its protocol behavior. The modifications are confined to the Raft messaging flow, specifically the points where client requests enter the system, where leader replicates log entries, and where followers receive and apply them.

We integrate UniCache at four points in etcd/raft:

1. The propose path
2. The leader append path
3. The follower receive path
4. The commit/apply path

These paths correspond to distinct modules in the etcd/raft codebase. The propose path resides in the raft/node.go module and represents the points where data from client requests first enters the system via Node.Propose(). Both the leader append and follower receive paths are located in the raft/raft.go module, which implements the core protocol behavior of Raft, handling leader replication and followers log updates. Finally, the commit/apply path is integrated at the point where RawNode.Ready() is called. RawNode exposes batched Entries to persist and CommittedEntries to apply. Caches are updated deterministically from CommittedEntries, the application then applies them.

The following sections detail each integration point and how UniCache modifies its execution flow.

5.2 Protobuf Encoding

Protocol Buffers (protobuf) is a language-neutral, platform-independent mechanism for serializing structured data. It encodes data into a compact binary format using field numbers, wire types, and values, allowing efficient serialization and deserialization while preserving structure and semantics.

Each field in a protobuf message is represented by a tag, which encodes both the field number and the wire type. The field number uniquely identifies the field within the message definition, while the wire type specifies how the data for that field is encoded (for example, as a variable-length integer or as a byte sequence). The way the data appears after the tag depends on the wire type. For most types such as varints or fixed-size integers, the value follows directly. However, for length-delimited fields (wire type 2), the tag is followed by a length prefix that indicates the number of bytes in the field's value, and then by the actual data bytes. This length prefix is used for strings, byte arrays, embedded messages, and packed repeated fields. In figures 5.1 and 5.2, both the Key and Value fields are encoded as length-delimited strings, which is why each field includes a tag, a length, and the corresponding byte representation of its value.

To illustrate, consider a simple key-value object:

KVOBJect: { Key: "any", Value: "val" }

Figure 5.1: Original Key-Value before protobuf serialization.

When serialized using protobuf, each field is represented by its tag, length, and value. The Key and Value fields are encoded as length-delimited strings, with their respective tags (10 and 18 in this example) followed by the length of the string and its byte representation.

In protobuf, a tag is computed as:

$$Tag = (FieldNumber \ll 3) | WireType$$

where $\ll 3$ denotes a bitwise left shift by 3 bits. This compact representation combines both metadata and content efficiently. Figure 5.2 shows how the key-value pair after it has been serialized.

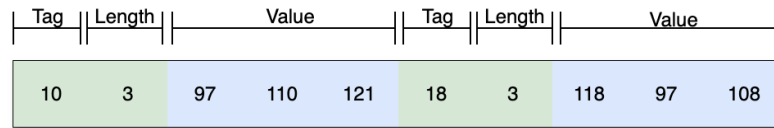


Figure 5.2: Key-Value after serialized using protobuf.

In our Raft UniCache layer, log entries are preprocessed before replication: frequent byte sequences are rewritten in place so that the original length-delimited bytes field (wire type 2) is replaced by a varint field (wire type 0) carrying the identifier. This is done by rewriting the field's tag, changing both the field number and the wire type as needed and for length-delimited fields, removing the length prefix and payload since the value is now a varint. Followers apply the inverse transform using the shared cache, they read the varint identifier, look up the original bytes, and restore the field by rewriting the tag back to wire type 2 and re-emitting the length and value bytes.

Figure 5.3 illustrates this swap, the key-value pair is shown after processing, where the “key” field is represented by its compact identifier instead of the original bytes, reducing space while preserving full protobuf validity and allowing exact reconstruction at the followers.

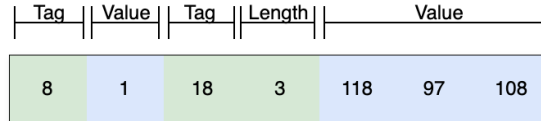


Figure 5.3: Key-Value serialized using protobuf. Field number 1 (key) has been replaced with its identifier.

5.3 Configuration

The module is configured with capacity C , field number f , and the callback function `minCacheVersion()` that returns the lowest cache version across nodes in the cluster.

5.4 Propose Path

The `EncodeData` method is invoked inside the raft state machine's `Step` function, at the point where `MsgProp` messages are processed. When the

application calls `Node.Propose()`, the data is placed on an internal proposal channel. The raft state machine picks up the message in `Step`, where `EncodeData` is called on each entry. If the key is found in the local cache, the entry's data is replaced with the compressed form and the assigned ID is stored in the `EncodedID` field. If not, the entry is left unchanged. The message is then passed to the role-specific step handler: on a follower this forwards the (possibly encoded) proposal to the leader, reducing follower-to-leader traffic. On the leader, it proceeds directly to the append path.

Algorithm 1 : EncodeData

On any node that accepts a client request, before proposing data d , `EncodeData` inspects the configured Protobuf field f . If the field is present in cache, the function rewrites the field f from Bytes to Varint(ID). If the field is absent or unknown, the payload is left unchanged. This opportunistic step reduces the client $\rightarrow$ leader traffic when the key is already known.

Input: payload bytes d , Protobuf field number f

Output: (d', id) where d' may have field f replaced by `Varint(id)`

$field \leftarrow \text{FIELD_BYTES}(d, f)$

if $field = \perp$ **then**

return $(d, 0)$

$id \leftarrow \text{REVERSE_CACHE}[k]$

if $id = \perp$ **or** $id \notin \text{CACHE}$ **then**

return $(d, 0)$

$d' \leftarrow \text{REPLACE_FIELD}(d, f, \text{Varint}(id))$

return (d', id)

This logic affects the size of the data replicated by etcd/raft. Both leader and follower code are located inside the `raft/raft.go` module to ensure encoded entries are decoded before appended to the log in order to preserve the API.

5.5 Leader Append Path

The leader append path resides inside `raft/raft.go` within the `stepLeader` and `appendEntry` functions, responsible for retrieving proposed entries and appending them to the leader log. Entries to be replicated are later created from the leaders log by slicing up the next indexes to send to each follower.

Since we want to keep the API of the library untouched, always exposing full `Entries` and `CommittedEntries` to the application, we introduce a pending buffer from which we instead create entries to send to followers. This way we

can append entries that may be encoded to the pending buffer, while decoding them before appending them to the actual leader log.

This works in the following way: the leader calls `SafeEncode` for each entry its about to append to its log. `SafeEncode` returns two byte slices: `maybeEncodedData` and `fullData`. `MaybeEncodedData` can be encoded or decoded depending on whether the entry is safe to send encoded. The following conditions (mentioned in design section) determine the safety of sending an encoded entry:

1. $appendIdx(key) - lastIdx(key) \leq capacity(cache)$
2. $minCacheVersion(followers) \geq addedIdx(key)$

If both conditions are met, `maybeEncodedData` will be returned as encoded bytes, which will be appended to the pending buffer for replication while the `fullData` will be appended to the raft log. However, if the conditions aren't met, `maybeEncodedData` will be returned as full bytes and the leader will replicated the full entry.

Algorithm 2 : SafeEncode

Input: payload bytes d , append idx i , encoded ID id

Output: (`maybeEncodedData`, `fullData`)

if $d = \emptyset$ **or** $id = 0$ **then**

return (d, d)

if $id \in CACHE$ **then**

$(a, \ell, k) \leftarrow (ADDEDIDX[id], LASTIDX[id], KEY[id])$

if $(i - \ell \leq C)$ **and** $(MINCACHEVERSION() \geq a)$ **then**

$fullData \leftarrow REPLACEFIELD(d, f, k)$

$encodedData \leftarrow d$

return ($encodedData, fullData$)

$(a, \ell, k) \leftarrow (ADDEDIDX[id], LASTIDX[id], EVICTEDKEY[id])$

$full \leftarrow REPLACEFIELD(d, f, k)$

return ($fullData, fullData$)

Algorithm 3 : AppendEntry

In the AppendEntry function, the leader calls SafeEncode and appends maybeEncodedData to the pending buffer. It also uses the fullData returned by SafeEncode to restore any encoded entries before appending them to its local log.

Input: proposed entries array E , current term T , last log index ℓ

Output:

```

1  $W \leftarrow$  new array of length  $|E|$ 
2 for  $i \leftarrow 0$  to  $|E| - 1$  do
3    $e \leftarrow E[i]$ 
4    $e.\text{Index} \leftarrow \ell + 1 + i$ 
5    $(\text{maybeEncoded}, \text{fullData}) \leftarrow \text{SAFEENCODE}(e.\text{Data}, e.\text{Index}, e.\text{EncodedID})$ 
6   if  $\text{maybeEncoded} = \perp$  then
7      $\text{maybeEncoded} \leftarrow \text{fullData}$ 
8
9    $E[i].\text{Data} \leftarrow \text{fullData}$ 
10   $W[i] \leftarrow e;$ 
11   $W[i].\text{Data} \leftarrow \text{maybeEncoded}$ 
12 if  $|W| > 0$  then
13   $\text{PENDING.TRUNCATEANDAPPEND}(W)$ 
  // Continue the original AppendEntry function

```

The safety check ensures unsafe encoded entries are never replicated and the pending buffer enables the leader to always append full entries to its log.

If SafeEncode cannot resolve the encoded ID, it returns nil for both the encoded and full data, the leader drops the entire proposal. This can occur when a follower encodes a proposal with a cache ID that has since been evicted and purged from the leader's retention buffer. Rather than appending an encoded entry that followers could not decode, the leader rejects the proposal and the client-application will handle the re-proposing. This guarantees that every entry written to the log and pending buffer carries data that all followers can reconstruct.

5.6 Follower Receive Path

Followers process AppendEntriesRPCs in raft.Step(), this is where new entries are appended to the log. We insert our DecodeEntry function immediately before the follower appends received entries to its log. This guarantees that

the followers logs remain identical to the leader's log, containing only full entries.

The `EncodedID` field on each entry is preserved after decoding. Although the payload is restored to its full form, the integer ID that was assigned during `EncodeData` remains on the entry struct. When these entries later reach the commit path 5.7, `UpdateCache` uses this ID to take the zero-allocation fast path 5, skipping field extraction entirely.

Algorithm 4 : DecodeEntry

`DecodeEntry` checks field f of the payload. If it carries `Varint(ID)`, the follower looks up the ID in its cache and restores the field back to `Bytes` using the cache. This procedure ensures followers only store full entries in their logs at all times. The leader checks detailed in the previous section prevents IDs from being unexpectedly missing.

Input: entry e , field f

Output: e'

if `FIELDTYPE`($e.Data, f$) = `Bytes` **then**

$\perp$ **return** e

$id \leftarrow \text{READVARINT}(e.Data, f)$

$field \leftarrow \text{REVERSECACHE}[id]$

$decoded \leftarrow \text{REPLACEFIELD}(e.Data, f, field)$

$e.Data \leftarrow decoded$

return e

The placement of `DecodeEntry` at this point isolates caching to the network layer. From this point on, nodes will only handle full entries, with the addition of using these entries to update caches deterministically in the commit path.

5.7 Commit Path

The commit path is where each node updates its local cache from newly committed entries. Inside `commitTo()`, the library calls `BatchUpdateCache` on every batch of entries whose commit index advances. By this point all entries are already fully decoded, the leader restored them via `SafeEncode` before appending to its log, and followers restored them via `DecodeEntry` before appending to theirs.

For each entry, `UpdateCache` first checks whether `EncodedID` $\neq 0$. If so, the entry's key is already in cache and the update reduces to a direct ID lookup. This path dominates at steady state: followers receive entries with `EncodedID` set by the proposing node, and the leader's own proposals

carry it from `EncodeData`. Only entries with previously unseen keys (`EncodedID = 0`) fall through to the slower path that extracts the field bytes and allocates a new cache mapping.

Algorithm 5 : UpdateCache

Otherwise, for a previously unseen value in field f , the node allocates. When entries become committed, each node invokes `UpdateCache`. If the entry carries a non-zero `EncodedID` (set during `EncodeData`), the node skips field extraction and updates the existing cache entry directly by ID—a zero-allocation fast path. a fresh ID and records both `addedIdx` and `lastIdx` as the commit index of the entry. For a known key, it refreshes the LRU position and updates `lastIdx`. Because caches are updated *only* from committed entries, every node converges to the same key $\leftrightarrow$ ID mappings in their caches.

Input: entry e , field f

Output:

Output:

```

if  $e.\text{EncodedID} \neq 0$  and  $e.\text{EncodedID} \in \text{CACHE}$  then
     $id \leftarrow e.\text{EncodedID}$ 
     $\text{LASTIDX}[id] \leftarrow \max(\text{LASTIDX}[id], e.\text{Index})$ 
     $\text{LRU}.\text{MoveToFront}(id)$ 
     $\text{MAYBEVICTLRU}(e.\text{Index})$ 
    return
 $\text{fieldvalue} \leftarrow \text{FIELDBYTES}(e.\text{Data})$ 
if  $\text{fieldvalue} \notin \text{REVERSECACHE}$  then
     $id \leftarrow \text{NEXTID}()$ 
     $\text{CACHE}[id] \leftarrow (\text{fieldvalue}, \text{ADDEDIDX} = e.\text{Index}, \text{LASTIDX} = e.\text{Index})$ 
     $\text{REVERSECACHE}[\text{fieldvalue}] \leftarrow id$ 
     $\text{LRU}.\text{INSERTFRONT}(id)$ 
     $\text{MAYBEVICTLRU}(e.\text{Index})$ 
else
     $id \leftarrow \text{REVERSECACHE}[\text{fieldvalue}]$ 
     $\text{LASTIDX}[id] \leftarrow \max(\text{LASTIDX}[id], e.\text{Index})$ 
     $\text{LRU}.\text{MoveToFront}(id)$ 
     $\text{MAYBEVICTLRU}(e.\text{Index})$ 

```

The `Ready()` interface continues to return fully decoded data, preserving the etcd/raft API.

5.8 Eviction and Retention

The active cache is bounded by capacity C and maintained with an LRU list. When inserting a new field f would exceed C , the least recently used value of the cache is evicted from the active cache. To tolerate lagging followers, the leader retains evicted values to a bounded retention buffer, introduced in 4.2.4. This lets the leader reconstruct entries to full bytes if an encoded proposal references an evicted ID.

Algorithm 6 PurgeEvicted

Input:

Output:

while $|EVICTED| > evictedCapacity$ **do**

$id \leftarrow EVICTED.FRONT()$

$EVICTED.POPFRONT()$

delete $EVICTED[id]$

Chapter 6

Experiments

This chapter presents an experimental evaluation of the modified system, focusing on the performance implications of introducing UniCache to etcd/raft and reducing follower -> leader bandwidth.

6.1 Setup

We evaluate the work in a minimal gRPC key-value service implemented on top of the etcd/raft library. The service exposes a Put(key, value) RPC. Upon receiving client requests, a node proposes a raft log entry and blocks the RPC until the entry commits, and then returns success. This closed-loop design ensures client load adapts to system capacity and that measured latencies reflect end-to-end commit time rather than queueing effects. The implementation uses Protobuf-encoded payloads and preserves the standard etcd/raft interface, encoding affects on-wire replication only, each node's local log remains fully decoded.

Three configurations will be compared:

1. Raft (no caching)
2. UniCache
3. RepliCache

Raft establishes the baseline without caching. UniCache demonstrates the effects of encoding at the leader only before replication. RepliCache shows the impact of followers forwarding encoded data from client requests to leader.

6.2 Experimental Setup

6.2.1 Distributed Experiments

To measure the impact of followers utilizing caching to propose encoded entries, distributed benchmarks are run on a raft cluster of 5 nodes spread across 5 regions, eu-west1, us-central1, asia-northeast1, southamerica-east, and south-australia. We also deploy 5 go-ycsb clients in each region, connected to the node in that region. All nodes are deployed on e2-standard-4 instances with 4 vCPUs, 16GB of memory, and 8Gbps maximum network bandwidth, running on Google Cloud Platform. We have each client running 3k concurrent client threads for 30s, resulting in a load of 15k concurrent requests distributed evenly across the cluster.

6.2.2 Workload

YCSB will be used to generate workload and benchmark the system. YCSB is an open-source benchmarking tool used to evaluate and tune the performance of database systems by enabling customization of workload patterns and performing metrics collection of latency and throughput.

The original YCSB library is implemented in Java and doesn't have a raft binding implemented. Since the Raft application used to implement RepliCache is written in Go and exposes a gRPC API we will use a Go fork of YCSB. This fork doesn't have a raft binding implemented either, so one will have to be created.

A 100% write workload of key-value pairs will be generated against the gRPC API. Clients issue put requests with configurable concurrency (number of client threads) and payload size. To vary the payload size, we specify the desired size of the key field of the put operation during configuration.

Throughput is collected for each run and the average of all runs for each parameters will be recorded and plotted.

6.2.3 Compression Rate

Raft replication traffic will be measured at the OS level using Linux nftables. Host-level byte counters capture on-wire bytes without instrumenting the code. For each run we install rules matching the Raft peer ports, reset counters, execute the go-YCSB workload, and then record the byte totals.

6.3 Results

6.3.1 Impact of Forwarding Encoded Entries

Figure 6.1 compares the throughput of Raft, UniCache, and RepliCache across payload sizes of 1 kB, 2 kB, and 4 kB, and across multiple cache hit ratios. Both encoding modes consistently outperform Raft for all evaluated payload sizes, demonstrating that encoding effectively reduces replication bandwidth and improves commit throughput. We also observe that across all settings (data size and hit rate), RepliCache outperforms the original UniCache, demonstrating the impact of followers proposing encoded entries.

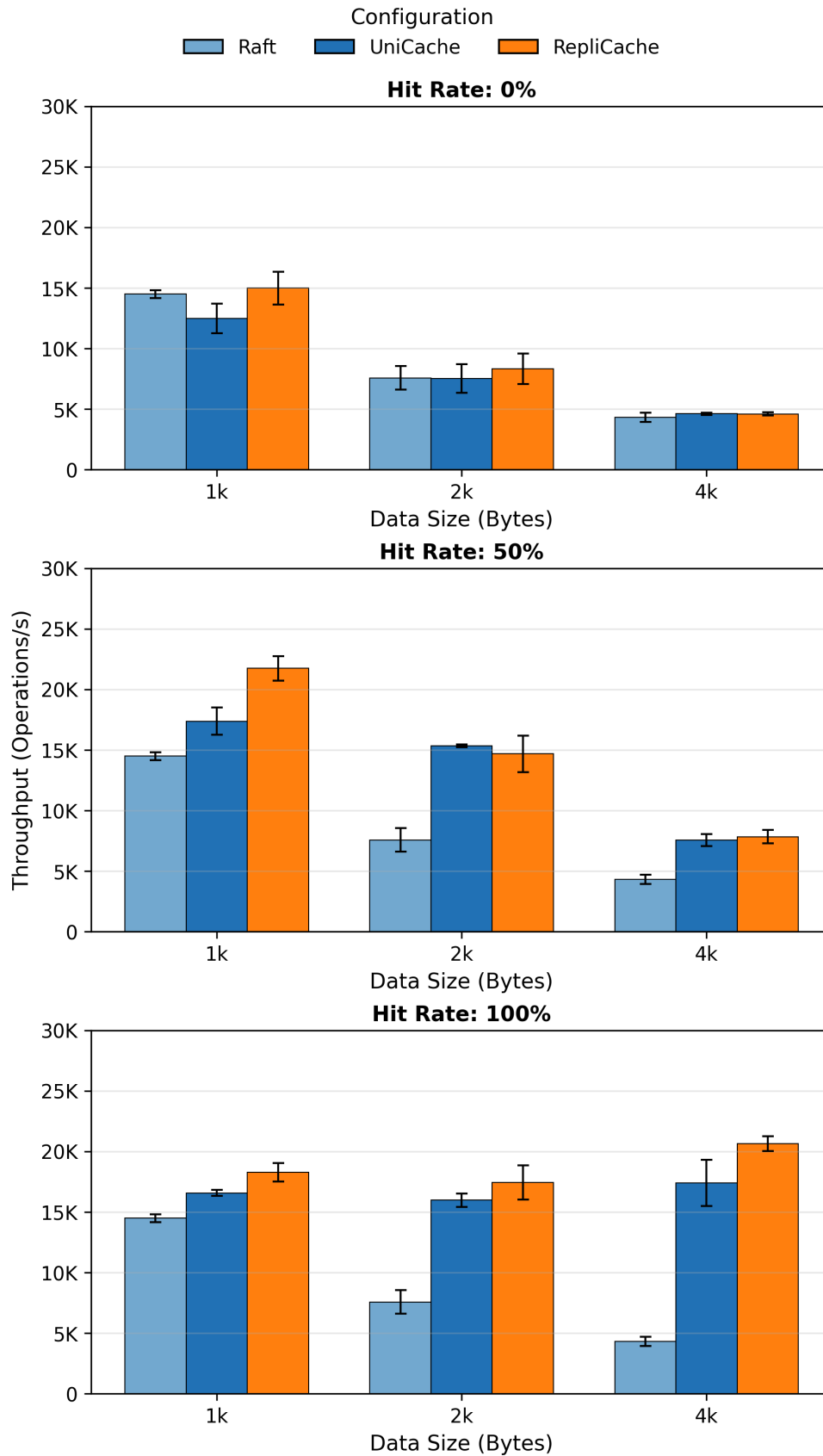


Figure 6.1: Throughput comparison of all modes where each node and client is placed in their own region (europe-west, us-central, asia-northeast, southamerica-east, australia-southeast)

For Raft, throughput decreases steadily as payload size grows from roughly 15k ops/s at 1 kB to 7.5k ops/s at 2 kB, and further to 5k ops/s at 4 kB. This decline reflects the increasing cost of transmitting larger values across regions. In comparison, UniCache steadily delivers around 16k ops/s to 17k ops/s for all data sizes and hit rates, except for 4 kB on 50% hit rate. This represents an improvement by more than 3x over the baseline at 4 kB. The throughput advantage shows how significantly bandwidth reduction affects performance under WAN latency. RepliCache achieves the highest throughput across all configurations, with the largest benefits appearing when cache hit rates are high. At 0% hit rate, the variants evaluated perform similarly, with the throughput decreasing for larger data sizes. As the hit rate increases to 50%, the effects of encoding becomes apparent. Both UniCache and RepliCache outperforming Raft by around 20% for data sizes of 1 kB. This is also clear for data sizes of 2 kB and 4 kB, where both encoding modes outperform Raft by 2x. At 100% hit rate, we observe the most significant gains. At 1 kB the gain, where RepliCache Encode delivers 17.5k ops/s for 1 kB and 2 kB, and a peak of 20.5k ops/s for 4 kB. This is the highest relative gains observed in terms of throughput: 4x gain compared to the 5k ops/s recorded for Raft. RepliCache also shows improvements in throughput over UniCache by 5-20% for the 100% hit rate scenario and up to 20% in the 50% hit rate scenario. This demonstrates the impact enabling followers to utilize the cache before sending proposals.

6.3.2 Leader in majority region

Figure 6.2 shows the throughput of a cluster where the leader is co-located with a majority of nodes in a single region, leading to much lower commit-latency. In this configuration, the Raft baseline performs significantly better than in the fully distributed case. This occurs because the leader can reach a commit quorum with the two local followers in its own region, meaning cross-continent communication is not needed to reach a commit.

At 0% hit rate, the caching variants both show a performance regression compared to Raft. We hypothesize this is related to memory allocation pressure from the caching operations, this is discussed further in section 6.5 and chapter 7. However, as the hit rates reach 50%, RepliCache starts to show improvements over UniCache for all data sizes. For 100% hit rate this improvement is mostly apparent when replicating data sizes of 2 kB. This demonstrates that while local quorums reduce the need for compression, the bandwidth savings from RepliCache still provide a net gain over UniCache, especially when handling larger entries.

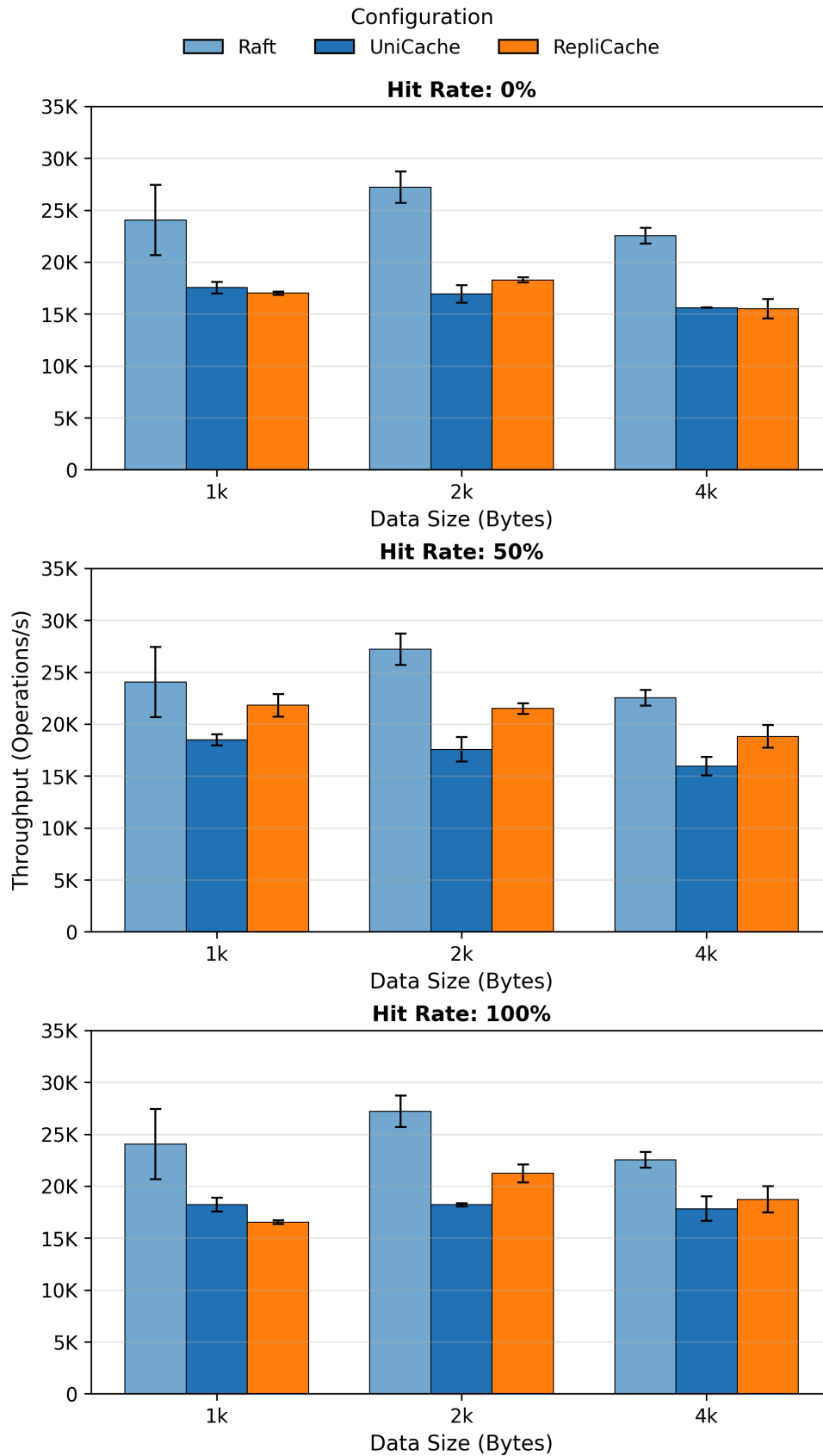


Figure 6.2: Throughput comparison of all modes where the leader is placed with a majority of nodes in eu-west, and the remaining two nodes in their own regions, one in southamerica-east and the other in asia-northeast.

The throughput performance regression for UniCache and RepliCache in this configuration is offset by the significant reduction in replication traffic details in section 6.3.4. While Raft achieves higher operations per second, it does so while transmitting redundant full-sized data across the network. Figure 6.4 shows that RepliCache lowers the amount of data sent by each replica when the hit rate is above 0%. This reduction in cross-region replication traffic can be translated directly into lower data transfer costs. RepliCache offers a trade-off between cost and throughput. While Raft maintains higher operations per second, RepliCache significantly reduces cross-region data transfer.

6.3.3 Real World Workload

To measure UniCache under realistic conditions, we utilized a subset of the Twitter Memcached (Twemcache) workload. The workload is a production trace of key-value operations characterized by high temporal locality and varying payload sizes ranging from 50 to 900 bytes. [29] This workload is characterized by varying key-value sizes (for this particular subset, sizes vary between 50 and 900 bytes) and high temporal locality. Similar to the experiment run in 6.3.1 each node is deployed in a separate region with its own client generating the workload. Figure 6.3 shows that both UniCache and RepliCache is outperformed by Raft on this workload, with RepliCache showing the largest regression with approximately 30% lower throughput than Raft. This outcome is expected given the payload characteristics of the Twemcache trace: with sizes between 50 and 900 bytes, the volume of data transmitted per operation is small enough that network transmission is not the primary bottleneck, even across regions. The encoding and decoding overhead in CPU time and memory allocations therefore dominates any bandwidth savings. This establishes a clear boundary condition for RepliCache, it is only beneficial when payloads are large enough for cross-region transmission to be the bottleneck. This condition holds true for the synthetic workload used in 6.3.1 but not for this trace.

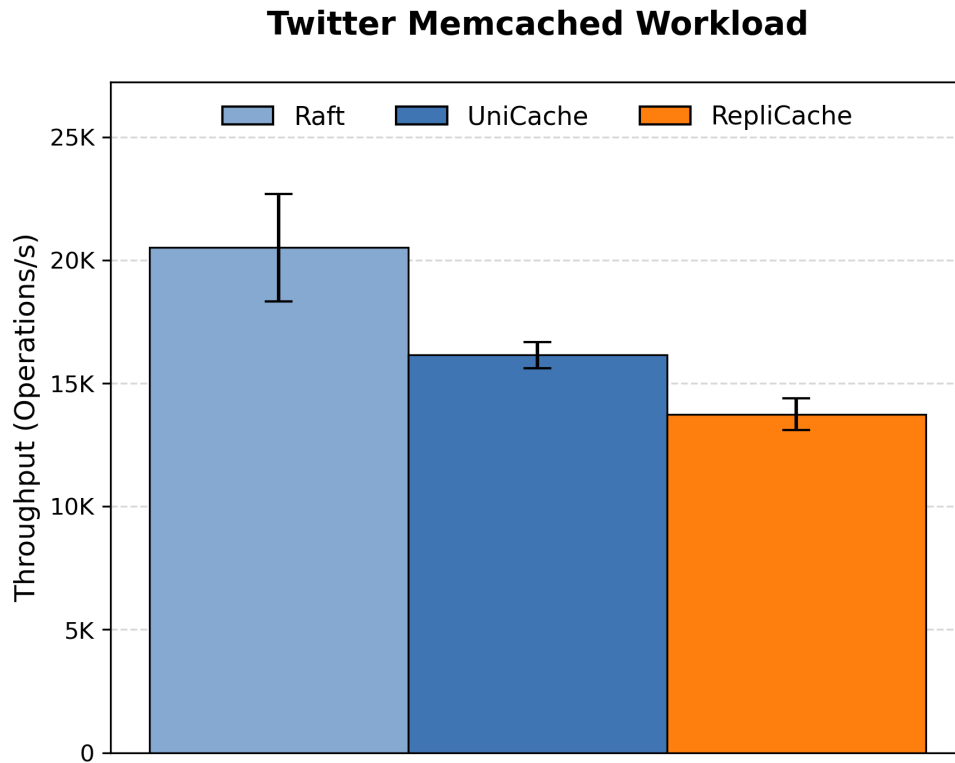


Figure 6.3: Throughput comparison of UniCache at 75% hit rate vs Raft with each node in their own region, and one client in the leader region driving the workload.

6.3.4 Compression

To evaluate the benefits of the follower-side caching of RepliCache, we quantify the byte savings achieved through 500k operations across 5 clients. This experiment highlights the benefits of enabling followers to perform compression, which reduces the volume of data on the forwarding path to the leader and further optimizes network efficiency.

Figure 6.4 summarizes the total data transmitted by the whole cluster, followers and leader for the different hit rates and payload sizes evaluated.

At a 50% hit rate, RepliCache reduces the total cluster bytes sent by approximately 50% compared to the 0% hit rate scenario (Raft). While UniCache also provides substantial savings at this rate, RepliCache further lowers the volume by an additional 15% due to followers' ability to encode client proposals.

At a 100% hit rate, the reduction is even more dramatic, with RepliCache

transmitting 5% to 20% of the data sent in the 0% scenario. Comparing to UniCache specifically, RepliCache sends between 20% to 50% less data.

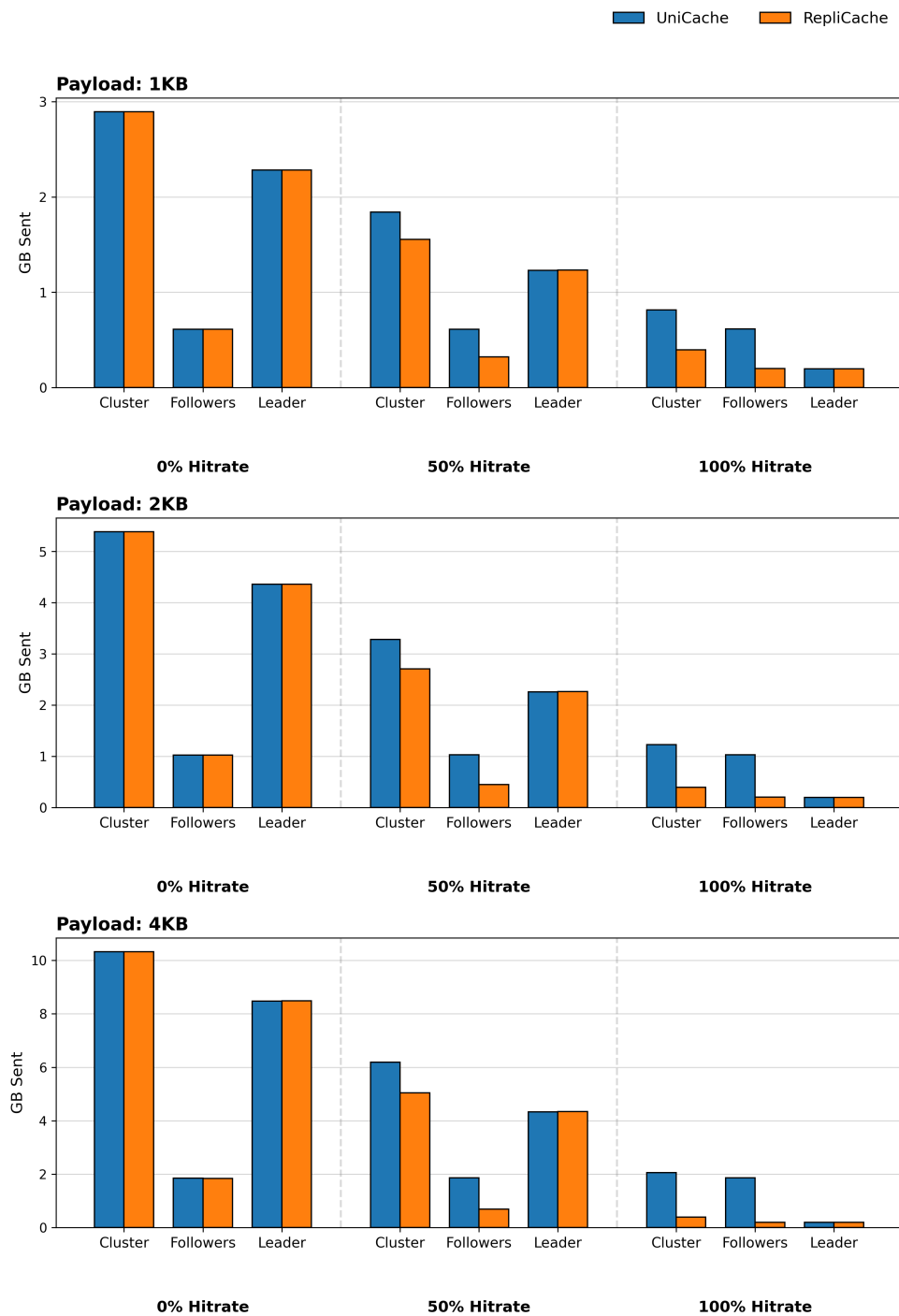


Figure 6.4: Comparison of UniCache vs RepliCache: total bytes sent by cluster, followers, and leader for payloads 1 kB, 2 kB and 4 kB and hit rates 0%, 50% and 100%. (lower is better)

These reductions explain the throughput improvements observed in the previous experiments. Because caching replaces recurring data fragments with compact identifiers, the effective payload size per replication round is significantly smaller. Lower transmission volume directly reduces cross-region latency and leader egress bandwidth, enabling the system to commit entries faster and sustain higher throughput.

6.4 Leader Transitions

To answer research question 2:

How can caches be synchronized between nodes without sending the cache?

The elected leader has the most up to date log by the design of Raft, it also has the most up to date cache and no cache synchronization is needed.

The setup used in 6.3.1 is used for for this experiment, with the exception of the leader node being terminated at $t=22.5s$, triggering a leader election. The remaining 4 nodes elect a new leader and then continues serving and replicating client requests.

An observation from Figure 6.5 is that the aggregate throughput increases after the leader transition for both Raft and RepliCache. This throughput depends on in which region the newly elected leader resides. If a leader with shorter RTT to reach a quorum the throughput increases, and vice versa.

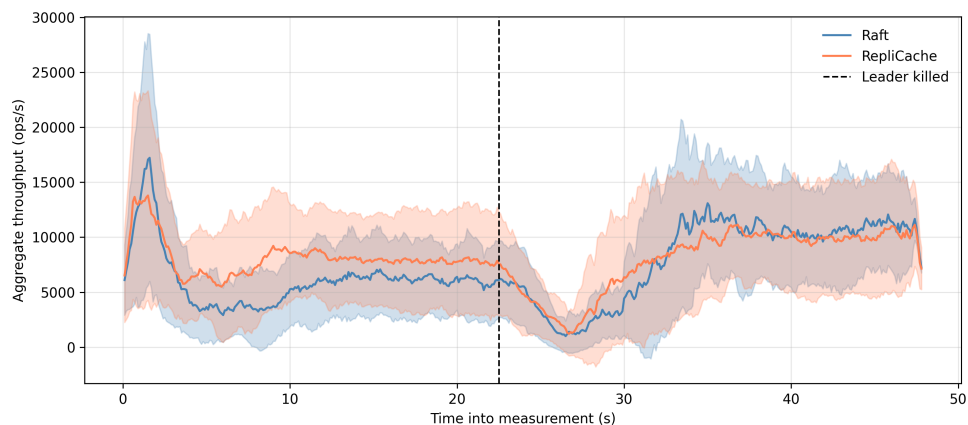


Figure 6.5: Throughput as a function of time. Leader change is triggered at time $t=22.5s$. Shaded region shows ± 1 std. deviation across runs.

This experiment demonstrates that RepliCache adds no extra synchronization on top of Raft during leader elections.

6.5 Microbenchmark Analysis

To isolate per-operation cost, we run Go microbenchmarks on the same e2-standard-4 GCP machines use throughout this section. Each benchmark operates on a single entry with payload sizes of 1, 2, and 4 kB at cache hit rates of 0%, 50%, and 100%. We benchmark the four core operations of RepliCache: `EncodeData`, `DecodeEntry`, `SafeEncode`, and `UpdateCache`. The benchmarks are run five times per configuration (data size and hit rate).

Operation	Hit Rate	ns/op	B/op	allocs/op
EncodeData	0%	1,505	4,096	1
	50%	1,486	4,104	2
	100%	2,596	4,112	3
DecodeEntry	0%	55	0	0
	50%	865	2,448	0
	100%	2,270	4,864	1
SafeEncode	0%	5.1	0	0
	50%	773	2,448	0
	100%	2,757	4,864	1
UpdateCache	0%	2,528	4,108	1
	50%	1,941	4,100	1
	100%	2,443	4,096	1

Table 6.1: Microbenchmark results at 4 kB payload (mean of 5 runs).

Table 6.1 presents the results at 4 kB; the 1 kB and 2 kB results follow the same pattern with proportionally lower costs. The results reveal two zero-allocation fast paths that dominate in common-case execution:

- `SafeEncode` at 0% hit: **5.1 ns, 0 allocs**. When no entry is encoded, a single field check returns immediately.
- `DecodeEntry` at 0% hit: **55 ns, 0 allocs**. A pre-scan for the encoded-field varint tag short-circuits before any payload processing.

These fast paths are significant because they guard the two hottest code paths in the replication pipeline: `SafeEncode` is called by the leader for every entry during `appendEntry`, and `DecodeEntry` is called by every follower on each received entry. When the cache has no encoded entries to process which is the case during a cold start or for non-recurring keys, both operations complete in under 100 ns with zero heap allocations.

The encoding and decoding operations become more expensive when keys are actively entering or leaving the cache. At 100% hit rate, `EncodeData` costs 2,6 ns with 3 allocations (copy + lookup + field replacement), and `DecodeEntry` costs 2,3 ns with 1 allocation (payload reconstruction). `UpdateCache` consistently allocates one object per call (4 kB for the field value copy), with cost ranging from 1,9 ns at 50% hit to 2,5 ns at 0% hit. The 50% case is cheapest because existing keys skip the LRU insertion path and only update metadata, while the 0% case pays full new-key bookkeeping cost.

In steady-state workloads with temporal key locality, most committed entries contain keys already in the cache. The leader's `SafeEncode` and each follower's `DecodeEntry` hit the zero-allocation fast paths, while `UpdateCache` performs a constant-cost metadata update.

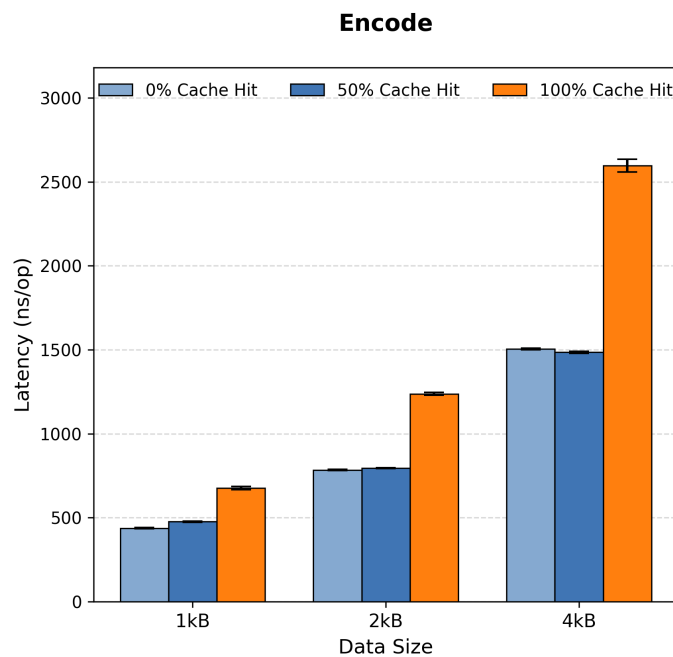


Figure 6.6: CPU overhead of the `EncodeData` function.

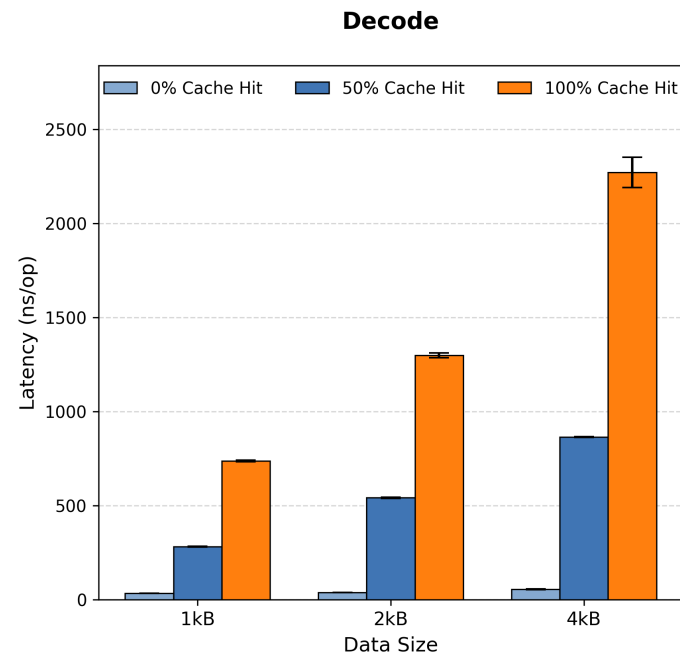


Figure 6.7: CPU overhead of the DecodeEntry function.

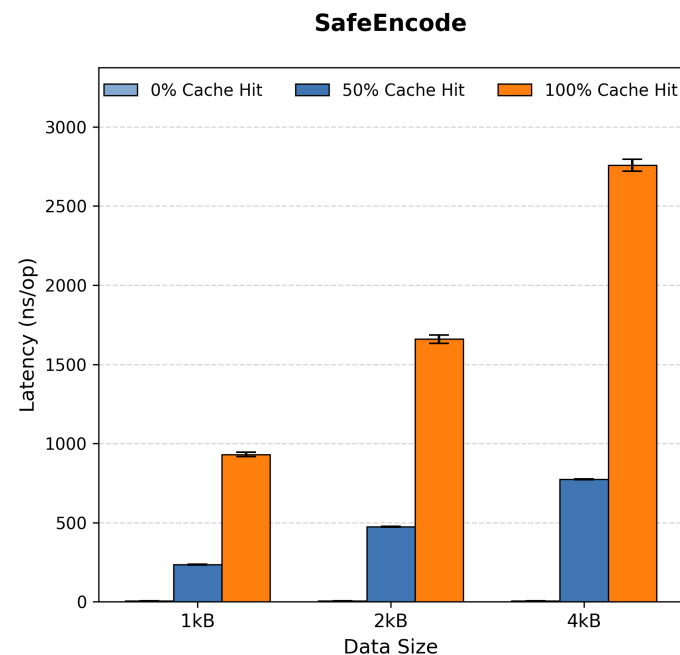


Figure 6.8: CPU overhead of the SafeEncode function.

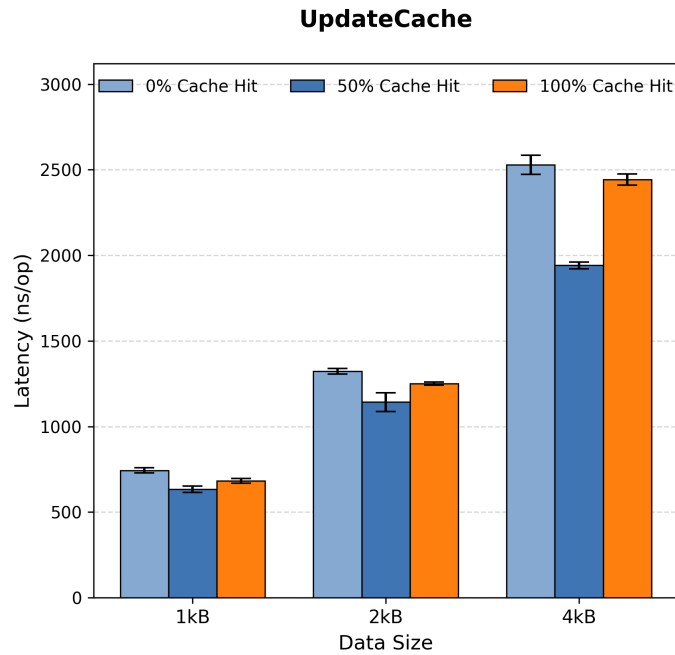


Figure 6.9: CPU overhead of the UpdateCache function.

The bar charts confirm the patterns from Table 6.1: overhead scales linearly with payload size, and the 0% hit rate scenario incurs near-zero cost for `DecodeEntry` and `SafeEncode` due to the early-exit fast paths. `EncodeData` retains a baseline cost even at 0% hit because it always performs a cache lookup to check for a matching key, regardless of whether encoding occurs. `UpdateCache` shows stable cost across hit rates, as it must extract and store the field value on every call.

To put these numbers into context, the absolute worst-case overhead introduced by any `RepliCache` function is 2.8 ns (`SafeEncode` at 4 kB, 100% hit). In a typical Raft deployment, a single `AppendEntries` round trip involves network latency of around 0.5 ms in LAN and over 100 ms in WAN. Because consensus is reached as soon as the leader has received a majority of acknowledgements, this single round trip represents the critical path for replication. Against this baseline, the maximum ~ 0.003 ms computational penalty added by `RepliCache` represents less than a 0.6% increase to the overall replication latency.

However, while individual operations are sub-microsecond to low-microsecond, these per-entry costs accumulate at high throughput within the single-threaded `node.run` goroutine that processes all Raft state transitions. This aggregate effect is analyzed via allocation profiling in Section 6.6.

6.6 Profiling

We collect Go `pprof` allocation profiles (`alloc_space`) during the 4 kB throughput experiments to understand where RepliCache’s overhead manifests at the system level. The `etcd/raft` library processes all state transitions in a single goroutine (`node.run` in `node.go`), which multiplexes proposals, inbound messages, configuration changes, and tick events via a `select` loop. This design simplifies correctness but means all RepliCache operations execute serially within this goroutine.

Metric	Raft		RepliCache	
	Leader	Follower	Leader	Follower
Total <code>alloc_space</code>	18.11 MB	16.02 MB	13.95 MB	12.58 MB
<code>node.run</code> cumulative	2.10 MB	1.37 MB	4.34 MB	3.32 MB
% of total allocs	11.6%	8.5%	31.1%	26.4%
<code>SafeEncode</code>	-	-	1.91 GB	-
<code>BatchUpdateCache</code>	-	-	66 MB	69 MB
<code>DecodeEntry</code>	-	-	-	1.81 GB
<code>EncodeData</code>	-	-	474 MB	415 MB
Live heap (after last GC)	5.98 GB	6.07 GB	3.98 GB	3.92 GB

Table 6.2: Allocation profile comparison (`alloc_space`, 4 kB payload). Leader and follower are shown separately. RepliCache operations only appear in the RepliCache column.

Table 6.2 reveals three key findings.

1. Garbage Collection is not the bottleneck. RepliCache allocates *less* total memory than baseline Raft (13.95 GB vs 18.11 GB) and follower (12.58 vs 16.02), because reduced message sizes decrease memory allocation in the transport layer. Live heap is also smaller (3.98/3.92 GB vs 5.98/6.07 GB), confirming that GC pressure is not the reason for the throughput regression observed in 6.2 and 6.3.
2. On the leader, `node.run` accounts for 31,1% of all allocations (4,34 GB), up from 11,6% in baseline Raft. The dominant cost is `SafeEncode` (1,91 GB), which produces the full-data backup for each encoded entry via `ReplaceProtoField`. Because `node.run` processes entries sequentially in a single goroutine, this per-entry

overhead accumulates linearly with throughput and limits the leader's capacity to process new proposals and heartbeats.

3. On followers, `BatchUpdateCache` allocates only 69 MB compared to 66 MB on the leader, nearly identical and negligible in both cases. This confirms that the `EncodedID` fast path (5) is effective on all nodes, reducing cache updates to zero-allocation lookups at steady state.

The dominant per-node costs are instead the protobuf field substitution operations: `SafeEncode` on the leader (1,91 GB) and `DecodeEntry` on followers (1,81 GB), both driven by `ReplaceProtoField`. Together with `EncodeData` (474 MB leader, 415 MB follower), these place `node.run` at 31,1% of total leader allocations (4,34 GB) and 26,4% on followers (3,32 GB).

This explains the regression pattern observed in 6.3.2 and 6.3.3: all `RepliCache` operations execute sequentially within the single-threaded `node.run` goroutine. While each operation is individually fast (6.5), the per-entry overhead accumulates linearly at high throughput, reducing the goroutine's capacity to process new proposals and heartbeats. In deployments where network latency is negligible, like the experiment in 6.3.2 or a LAN scenario, this CPU-bound overhead becomes the dominant cost and widens the gap between baseline Raft and `RepliCache`.

Chapter 7

Discussion

The experimental results demonstrate that caching substantially improves throughput without altering Raft’s semantics, the primary contribution of this work lies in the architectural extensions made to the original UniCache design. These improvements specifically address the bottlenecks associated with leader-centric encoding and protocol-specific synchronization.

The first research question sought to extend UniCache to support follower-side encoding. In the original UniCache implementation within Omni-Paxos, only the leader performed encoding. This meant all client proposals were routed through the leader as full, uncompressed data. With the RepliCache implementation in Raft, followers are allowed to compress client data before forwarding it to the leader. The results show that RepliCache outperforms UniCache by up to 20% in high hit ratio scenarios. This demonstrates that compressing client data before forwarding from follower to leader further optimizes UniCache, optimizing not just the leader but the entire cluster of leader-based consensus protocols.

However, the benefits of caching are not universal. As observed in 6.3.2 and 6.3.2, caching can degrade the performance when the leader is located in a region with majority of nodes or if the size of the data replicated is lower. For these low-latency environments and lower data sizes, the network is no longer the bottleneck. Allocation profiling (Section 6.6) shows that this degradation is not caused by garbage collection pressure, RepliCache actually allocates less total memory than Raft (13,95 GB vs 18,11 GB on the leader) and maintains a smaller live heap (3,98 GB vs 5,98 GB). Instead, the overhead stems from the single-threaded `node.run` goroutine that processes all Raft state transitions sequentially. RepliCache adds per-entry operations (`SafeEncode`, `DecodeEntry`, `EncodeData`) to this critical path. While

each is individually sub-microsecond (Section 6.5), at high throughput the aggregate cost accumulates linearly, reducing the goroutine’s capacity to process new proposals and heartbeats. In LAN deployments where network latency is negligible, this CPU-bound overhead becomes the dominant cost and widens the gap between baseline Raft and RepliCache. This would also explain why RepliCache isn’t showing larger throughput improvements compared to UniCache.

The second research question addressed how to synchronize caches without the cache exchange required by original UniCache. A central insight from our results is that cache determinism based on committed log entries is sufficient for safety, since in Raft, the leader election requires the new leader to have the most up to date log. This removes the need for Omni-Paxos prepare phase synchronization. The success of the RepliCache also reveals that Raft follower forwarding path can safely perform encoding before proposing data without affecting correctness. Encoding at the edge decentralizes bandwidth usage and balances load across the cluster, suggesting a general pattern: some leader responsibilities can be safely offloaded when their effects are verifiable by the leader. The RepliCache improvement makes UniCache better suited to systems like Raft, where any node may serve client requests.

Zone	United States	Europe	Asia	South America	Australia
United States	0.02	0.05	0.08	0.14	0.10
Europe	0.05	0.02	0.08	0.14	0.10
Asia	0.08	0.08	0.08	0.14	0.10
South America	0.14	0.14	0.14	0.14	0.14
Australia	0.10	0.10	0.10	0.14	0.08

Table 7.1: VM-to-VM data transfer costs on Google Cloud Platform (\$/GiB) [30]

Zone	United States	Europe	Asia	South America	Australia
United States	0.02	0.05	0.05	0.05	0.05
Europe	0.05	0.02	0.05	0.05	0.05
Asia	0.08	0.08	0.08	0.08	0.08
South America	0.16	0.16	0.16	0.16	0.16
Australia	0.08	0.08	0.08	0.08	0.08

Table 7.2: VM-to-VM data transfer costs on Microsoft Azure (\$/GB). [31]

The results presented in 6.4 established a clear relation between cache hit rates and reduction in replication traffic, which directly translates into financial savings in multi-region environments. On 50% cache hit rate, RepliCache achieved a 50% reduction compared to Raft, and a 15% reduction compared to UniCache. This translates to savings of 0.50 and 0.15 per dollar compared to Raft and UniCache respectively. For the 100% hit rate scenario, RepliCache achieves a reduction of 80-95% compared to Raft, and 50-80% compared to UniCache. This is translated into a financial win of between 0.80-0.95\$/GiB compared to Raft and 0.50-0.80\$ GiB compared to UniCache.

As the volumes of data replicated daily have been increasing during the latest years and data being a valuable resource, this financial benefit can be huge for companies using services such as Google Cloud Platform or Microsoft Azure.

As mentioned in 4, RepliCache makes use of a retention buffer on the leader, this introduces the risk of exploding leader memory on the leader when the cluster has slow followers. Seeing as the results show that RepliCache is just marginally better performing than UniCache, and mostly for large data sizes, it can be questioned if that small throughput gain alone motivates running it over normal UniCache.

Another benefit for running RepliCache mode over UniCache is the financial one, discussed in a previous paragraph. Cloud providers handle huge amounts of data every day, and since there is no performance degradation in using RepliCache, incentives for running the RepliCache variant over UniCache are high.

Chapter 8

Conclusions and Future Work

8.1 Conclusion

This work presented RepliCache, an extension to UniCache which enables followers to utilize caches to propose encoded entries, further optimizing bandwidth use for leader-based consensus protocols. By deriving caches from committed log entries without any additional coordination phase, all nodes serving client requests can perform encoding that the leader can then replicate without any changes to the protocol. This results in a substantial reduction in bytes sent and received by nodes during replication, using much less bandwidth while preserving full information.

Geo-distributed benchmarks show that these bandwidth savings translate into significant throughput improvements. The lowered volume of data sent between nodes could also reduce cross-region egress costs.

While our implementation is done in Raft, the approach can be applicable to any leader-based protocol without introducing any additional coordination phase or messages.

8.2 Future Work

Something to investigate further is integrating this implementation in a production system that already uses raft under the hood to measure if caching shows similar results. CockroachDB would be a natural candidate for this, as it's built on etcd Raft, the same library used for this work. It also comes with several load generators that can simulate client workloads of different types [32]. This could be achieved by swapping the raft dependency used and then benchmark using the built-in workloads simulating different scenarios.

A second direction of future work is to dynamically enable and disable caching based on runtime conditions. The evaluation shows that caching improves throughput when cross-region network latency dominates, but can degrade performance in low-latency deployments where the per entry CPU overhead in the single-threaded goroutine outweighs bandwidth savings. An adaptive mechanism could monitor signals such as observed network latency or payload size, and toggle encoding on and off accordingly. For example, caching could be activated when the measured round-trip to a quorum of followers exceeds a threshold or when the payload size is large enough to justify the encoding cost. Such a mechanism would allow a cluster of nodes to benefit from caching under favorable conditions while avoiding the throughput regression observed in scenarios where the network is not the bottleneck.

Thirdly, it could also be of value to benchmark smaller payloads and low latency scenarios.

Finally, this work focused on measuring the impacts of reducing the data sent during replication. What if data proposed contains multiple fields? It would be of interest to measure the implications of encoding multiple fields of a protobuf encoded byte array. For instance, if we have a protobuf definition with 10 fields, and we declare 8 of those fields "cacheable", how does the fact that we can "switch out" multiple fields affect the performance of the protocol? Does the encoding step become heavier, since we are switching out more fields per entry replicated, or do we get similar results?

References

- [1] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “Cockroachdb: The resilient geo-distributed sql database,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, 2020. doi: 10.1145/3318464.3386134. ISBN 9781450367356 p. 1493–1509. [Online]. Available: <https://doi.org/10.1145/3318464.3386134> [Pages 1, 7, and 9.]
- [2] “TiKV: A highly scalable, low latency, and easy to use key-value database,” <https://tikv.org/>, 2024, accessed: 2025-05-26. [Pages 1 and 9.]
- [3] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, “Spanner: Google’s globally distributed database,” *ACM Trans. Comput. Syst.*, vol. 31, no. 3, Aug. 2013. doi: 10.1145/2491245. [Online]. Available: <https://doi.org/10.1145/2491245> [Page 1.]
- [4] M. Burrows, “The chubby lock service for loosely-coupled distributed systems,” in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*, ser. OSDI '06. USA: USENIX Association, 2006. ISBN 1931971471 p. 335–350. [Page 1.]
- [5] F. Junqueira, B. Reed, and M. Serafini, “Zab: High-performance broadcast for primary-backup systems,” 07 2011. doi: 10.1109/DSN.2011.5958223 pp. 245 – 256. [Page 1.]

- [6] S. Ghemawat, H. Gobioff, and S.-T. Leung, “The google file system,” in *Proceedings of the nineteenth ACM symposium on Operating systems principles*, ser. SOSP '03. New York, NY, USA: ACM, 2003. doi: 10.1145/945445.945450. ISBN 1-58113-757-5 pp. 29–43. [Online]. Available: <http://doi.acm.org/10.1145/945445.945450> [Pages 1, 5, and 7.]
- [7] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The hadoop distributed file system,” in *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010. doi: 10.1109/MSST.2010.5496972 pp. 1–10. [Page 1.]
- [8] L. Lamport, “Paxos made simple,” *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001), pp. 51–58, December 2001. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/paxos-made-simple/> [Pages 1, 6, and 7.]
- [9] —, “The part-time parliament,” *ACM Trans. Comput. Syst.*, vol. 16, no. 2, p. 133–169, May 1998. doi: 10.1145/279227.279229. [Online]. Available: <https://doi.org/10.1145/279227.279229> [Pages 1, 6, and 7.]
- [10] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at google with borg,” in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys '15. New York, NY, USA: Association for Computing Machinery, 2015. doi: 10.1145/2741948.2741964. ISBN 9781450332385. [Online]. Available: <https://doi.org/10.1145/2741948.2741964> [Page 1.]
- [11] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIX ATC'14. USA: USENIX Association, 2014. ISBN 9781931971102 p. 305–320. [Pages 1, 3, 7, 8, 9, and 10.]
- [12] B. M. Oki and B. H. Liskov, “Viewstamped replication: A new primary copy method to support highly-available distributed systems,” in *Proceedings of the Seventh Annual ACM Symposium on Principles of Distributed Computing*, ser. PODC '88. New York, NY, USA: Association for Computing Machinery, 1988. doi: 10.1145/62546.62549. ISBN 0897912772 p. 8–17. [Online]. Available: <https://doi.org/10.1145/62546.62549> [Page 1.]

- [13] H. Ng, K. Wu, and P. Carbone, “Unicache : Efficient log replication through learning workload patterns,” in *Advances in Database Technology - EDBT* :, vol. 46, no. 3. OpenProceedings.org, 2023. doi: 10.48786/edbt.2023.39 pp. 471–477, this work has been supported by the Swedish Foundation of Strategic Research (Grant No.: BD15-0006), RISE AI, Google Cloud Research Credits Program, and the Wallenberg AI: Autonomous Systems and Software Program (Data-Bound NEST Project). [Pages 2, 3, 12, and 13.]
- [14] H. Ng, S. Haridi, and P. Carbone, “Omni-paxos: Breaking the barriers of partial connectivity,” in *Proceedings of the Eighteenth European Conference on Computer Systems*, ser. EuroSys ’23. New York, NY, USA: Association for Computing Machinery, 2023. doi: 10.1145/3552326.3587441. ISBN 9781450394871 p. 314–330. [Online]. Available: <https://doi.org/10.1145/3552326.3587441> [Pages 2, 3, 10, and 11.]
- [15] “etcd-io/raft: Raft library for maintaining a replicated state machine,” <https://github.com/etcd-io/raft>, 2025, accessed: 2025-07-07. [Page 3.]
- [16] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. USA: Addison-Wesley Publishing Company, 2011. ISBN 0132143011 [Pages 5 and 8.]
- [17] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *J. ACM*, vol. 35, no. 2, p. 288–323, Apr. 1988. doi: 10.1145/42282.42283. [Online]. Available: <https://doi.org/10.1145/42282.42283> [Page 6.]
- [18] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *J. ACM*, vol. 32, no. 2, p. 374–382, Apr. 1985. doi: 10.1145/3149.214121. [Online]. Available: <https://doi.org/10.1145/3149.214121> [Page 6.]
- [19] E. Androutaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger fabric: a distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys ’18. New York, NY, USA: Association for

- Computing Machinery, 2018. doi: 10.1145/3190508.3190538. ISBN 9781450355841. [Online]. Available: <https://doi.org/10.1145/3190508.3190538> [Page 7.]
- [20] F. B. Schneider, “Implementing fault-tolerant services using the state machine approach: a tutorial,” *ACM Comput. Surv.*, vol. 22, no. 4, p. 299–319, Dec. 1990. doi: 10.1145/98163.98167. [Online]. Available: <https://doi.org/10.1145/98163.98167> [Page 8.]
- [21] “etcd-io/etcd: A distributed, reliable key-value store for the most critical data of a distributed system,” <https://github.com/etcd-io/etcd>, 2025, accessed: 2025-05-26. [Page 9.]
- [22] “Overview: Dgraph,” <https://docs.hypermode.com/dgraph/overview>, 2025, accessed: 2025-05-26. [Page 9.]
- [23] “RethinkDB: the open-source database for the realtime web,” <https://rethinkdb.com/>, 2025, accessed: 2025-05-26. [Page 9.]
- [24] A. Rahut, V. Bhat, A. Sharma, Y. Shen, B. Pelc, C. Li, A. Haque, Y. Botadra, X. Wang, M. Percy, R. Yadav, Y. Matsunobu, A. Liang, I. Pozgaj, T. Asplund, A. Karp, L. Lou, and P. Goyal, “Myraft: High availability in mysql using raft,” in *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2024. doi: 10.48786/edbt.2024.64. [Online]. Available: <https://openproceedings.org/2024/conf/edbt/paper-199.pdf> [Page 12.]
- [25] A. Ganesan, R. Alagappan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Exploiting nil-externality for fast replicated storage,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, ser. SOSP ’21. New York, NY, USA: Association for Computing Machinery, 2021. doi: 10.1145/3477132.3483543. ISBN 9781450387095 p. 440–456. [Online]. Available: <https://doi.org/10.1145/3477132.3483543> [Page 12.]
- [26] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, “Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications,” *ACM Trans. Storage*, vol. 17, no. 4, Oct. 2021. doi: 10.1145/3483840. [Online]. Available: <https://doi.org/10.1145/3483840> [Page 12.]

- [27] L. Lamport, “Using time instead of timeout for fault-tolerant distributed systems.” *ACM Trans. Program. Lang. Syst.*, vol. 6, no. 2, p. 254–280, Apr. 1984. doi: 10.1145/2993.2994. [Online]. Available: <https://doi.org/10.1145/2993.2994> [Page 14.]
- [28] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan, “Driving cache replacement with ML-based LeCaR,” in *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. Boston, MA: USENIX Association, Jul. 2018. [Online]. Available: <https://www.usenix.org/conference/hotstorage18/presentation/vietri> [Page 14.]
- [29] Twitter, “Twitter memcached,” <https://github.com/twitter/twemcache>, 2026, accessed: 2026-02-01. [Page 43.]
- [30] Google Cloud, “Network pricing – virtual private cloud,” <https://cloud.google.com/vpc/network-pricing>, 2025, accessed: 2025-10-19. [Page 56.]
- [31] Microsoft Corporation, “Bandwidth pricing – microsoft azure,” <https://azure.microsoft.com/en-gb/pricing/details/bandwidth>, 2025, accessed: 2025-10-19. [Page 56.]
- [32] Cockroach Labs, “cockroach workload,” <https://www.cockroachlabs.com/docs/stable/cockroach-workload>, 2025, cockroachDB Docs, version v25.3.1. Accessed 2025-09-20. [Page 59.]

Appendix A

Tables

Table A.1: 5-node symmetric – 0% hit rate

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	1kB	–	994.8	1808.4
Raft	2kB	–	2006.0	4206.6
Raft	4kB	–	3201.0	6811.6
UniCache	1kB	0%	1197.1	2408.4
UniCache	2kB	0%	1924.6	4493.3
UniCache	4kB	0%	2811.9	6588.4
RepliCache	1kB	0%	1099.8	1569.8
RepliCache	2kB	0%	1700.9	4300.8
RepliCache	4kB	0%	3106.8	4812.8

Table A.2: 5-node symmetric – 50% hit rate

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	1kB	–	994.8	1808.4
Raft	2kB	–	2006.0	4206.6
Raft	4kB	–	3201.0	6811.6
UniCache	1kB	50%	815.6	1397.8
UniCache	2kB	50%	857.9	2155.5
UniCache	4kB	50%	1344.5	5574.7
RepliCache	1kB	50%	680.7	1485.8
RepliCache	2kB	50%	910.3	1995.8
RepliCache	4kB	50%	1440.8	5902.3

Table A.3: 5-node symmetric – 100% hit rate

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	1kB	–	994.8	1808.4
Raft	2kB	–	2006.0	4206.6
Raft	4kB	–	3201.0	6811.6
UniCache	1kB	100%	980.5	1503.2
UniCache	2kB	100%	891.9	1657.9
UniCache	4kB	100%	867.8	1637.4
RepliCache	1kB	100%	891.4	1267.7
RepliCache	2kB	100%	915.2	1648.6
RepliCache	4kB	100%	804.9	1181.7

Table A.4: Asymmetric majority – 0% hit rate

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	1kB	–	446.5	972.8
Raft	2kB	–	357.6	795.6
Raft	4kB	–	410.6	1157.1
UniCache	1kB	0%	731.1	1797.1
UniCache	2kB	0%	634.9	1568.8
UniCache	4kB	0%	603.6	1252.4
RepliCache	1kB	0%	580.6	1146.9
RepliCache	2kB	0%	586.2	1056.3
RepliCache	4kB	0%	679.4	1237.0

Table A.5: Asymmetric majority – 50% hit rate

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	1kB	–	446.5	972.8
Raft	2kB	–	357.6	795.6
Raft	4kB	–	410.6	1157.1
UniCache	1kB	50%	811.5	1267.7
UniCache	2kB	50%	581.1	1084.4
UniCache	4kB	50%	640.0	1248.3
RepliCache	1kB	50%	663.6	980.0
RepliCache	2kB	50%	536.6	1126.4
RepliCache	4kB	50%	544.8	959.5

Table A.6: Asymmetric majority – 100% hit rate

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	1kB	–	446.5	972.8
Raft	2kB	–	357.6	795.6
Raft	4kB	–	410.6	1157.1
UniCache	1kB	100%	810.5	1335.3
UniCache	2kB	100%	792.6	1211.4
UniCache	4kB	100%	828.7	1408.5
RepliCache	1kB	100%	903.7	1362.9
RepliCache	2kB	100%	695.3	1067.0
RepliCache	4kB	100%	524.8	1141.8

Table A.7: Trace workload (cluster26)

Config	Payload	Hit Rate	p50 (ms)	p99 (ms)
Raft	trace	–	760.8	1903.6
UniCache	trace	75%	963.6	2140.2
RepliCache	trace	75%	1165.3	2365.4

